

TEL Specification

TEL Specification Draft

Abstract

1. Status
2. Conformance Language
3. Overview
4. Character Encoding
5. Significant Characters and Terms
6. Root Structure
7. Interpreter Directive
8. Pragma
 - 8.1 Schema Identifier
 - 8.2 Schema Resolution
 - 8.3 Sigil Resolution
9. Lines, Margin, and Indentation
10. Keywords and Inline Atoms
 - 10.1 Keyword Characters
 - 10.2 Inline Atom Characters
 - 10.3 Phrase-Separation Rule
11. Comments and Remarks
 - 11.1 Comment
 - 11.2 Remark
12. Compound Tree Structure
13. Parent, Child, and Peer Relations
14. Source Atoms
15. Literal Atoms
16. Tabulations and Tabulated Blocks
 - 16.1 Tabulation Line
 - 16.2 Tabulated Block
17. Presentation Nodes
18. Presentation Model and Semantic Model
 - 18.1 Presentation Model
 - 18.2 Semantic Model
 - 18.3 Mapping Procedure
19. Schema-Governed Structure and Error Diagnosis
 - 19.1 Atom and Compound Interchangeability
 - 19.2 Positional Assignment
 - 19.3 Error Taxonomy
 - 19.4 Error Diagnosis
 - 19.5 Error Recovery
20. Schema Language
 - 20.1 Schema Validity Constraints
 - 20.2 Type Assignment Algorithm
 - 20.3 Schema Layering

- 20.4 BinTEL
- 20.5 The tel-schema Schema
- 20.6 Schema Construction from the Semantic Model
- 20.7 Identifier Grammars
- 20.8 Footnote: Inline-Atom Position Constraints
- 21. Validation
 - 21.1 Validators
 - 21.2 Request and Response
 - 21.3 Reporting and Span Resolution
 - 21.4 Helper Method Binding
 - 21.5 Built-in Validators
 - 21.6 Struct and Select Validators
- 22. Reserialization and Editing
 - 22.1 Comment Attachment and Editing
 - 22.2 Machine Operations
 - 22.3 Canonical Document Serialization
 - 22.4 Round-Trip Properties
 - 22.5 Concurrent Edit Composition
- 23. Invalidity Conditions
- 24. Formal Type System and Subtyping (Informative)
 - 24.1 Type Grammar
 - 24.2 Membership
 - 24.3 Subtype Relation
 - 24.4 Layer Composition Produces Subtypes
 - 24.5 The Liskov Substitution Theorem
 - 24.6 Consequences for Tooling
 - 24.7 What This Type System Does NOT Cover
- 25. Specification Status

TEL Specification Draft

Abstract

TEL is a line-oriented, tree-structured, typed data language designed for data that is read, written and maintained by *humans*, intelligent *agents* or deterministic *processors*.

TEL defines a **presentation model** that preserves comments, document structure and user data through programmatic round-trips, while permitting minor normalizations such as collapsing space-only blank lines to empty lines. A schema-driven **semantic model** ascribes types to every node in the tree. The two models are connected by a deterministic type-assignment algorithm. A companion specification, BinTEL, defines a compact binary encoding that provides an unambiguous serialization of the semantic model.

The design of TEL is motivated by the following goals:

- **Formatting preservation.** Machine edits should not disturb formatting, comments or whitespace on lines they do not semantically change, so that line-based version control produces minimal, meaningful diffs.

- **Minimal escaping.** Syntax conflicts between content and structure should be rare; literal and source atoms allow arbitrary content with no character escaping.
- **Strict, recoverable parsing.** Parsing is unambiguous and every error condition has a defined recovery strategy, so that a single mistake does not shadow subsequent errors.
- **Schema-driven typing.** Every node is typed by a schema. Validation, including string-level constraints, is an integral part of the format rather than an external layer.
- **Layered extensibility.** Schemas support append-only layering, enabling forwards-compatible extensions with clear compatibility semantics.
- **Human and machine editors.** The format is designed for direct human authorship, IDE tooling with immediate feedback, programmatic transformation, and AI-assisted editing alike.

1. Status

This document is a draft specification of TEL.

Where this draft contains FIXME notes, the corresponding behavior is not yet fully specified and **MUST NOT** be considered stable.

2. Conformance Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

3. Overview

TEL is a Unicode, character-based language for ordered, tree-structured data represented as strings, and typed according to a schema.

TEL presents data as an *ordered* tree, however an application consuming TEL **MAY** choose to assign meaning to sibling order, or **MAY** treat it as insignificant. In this respect, TEL is similar to XML.

TEL distinguishes between:

- a **presentation model**, which preserves comments, interpreter directives, pragma metadata, atom presentation form, most whitespace and document structure sufficiently for faithful reserialization, and
- a **semantic model**, which is derived from the presentation model using a schema.

This document specifies TEL source, its parsing into the presentation model, the definition of schemas and translation between presentation model and semantic model by means of a schema.

4. Character Encoding

TEL is defined over Unicode code points.

When written to a file, a TEL document **MUST** be encoded as UTF-8.

Line endings in a TEL document are governed by the following rules (literal atom payloads, defined in §15, are exempt from all of them):

1. The line-ending style is uniform across the entire document: either every line ends with LF, or every line ends with CR LF.
2. The **line-ending mode** is determined by the first CR or LF character in the document: if it is CR, the mode is **CRLF mode**; otherwise the mode is **LF mode**.
3. In CRLF mode, CR and LF may only appear as part of a CR LF line ending, except within literal atoms (**E121**).
4. In LF mode, CR may not appear anywhere in the document, except within literal atoms (**E121**).

LF mode is RECOMMENDED. Human authors **SHOULD** use LF endings but **MAY** use CRLF endings. Agents and processors **MUST** use LF endings when creating new documents, and **SHOULD NOT** change the line-ending mode of an existing document.

No Unicode normalization is required or implied. TEL is defined over the exact Unicode code points that appear in the serialized text.

A UTF-8 byte order mark **MUST NOT** appear in a TEL document (**E101**).

Visually misleading code points, such as zero-width characters, **SHOULD** be avoided. Control-heavy content **SHOULD** be avoided except where required. TEL is not intended primarily as a binary-data format, even though it can represent content containing non-printing code points.

5. Significant Characters and Terms

The following three characters have syntactic significance in TEL:

- U+000A LINE FEED (LF)
- U+0020 SPACE
- one other symbolic character designated as the **sigil**

The following definitions apply:

A **line** is a contiguous, potentially empty sequence of non-linefeed characters delimited by LF characters or by the start or end of the file. In CRLF mode (§4), the CR immediately preceding each delimiting LF is part of the line terminator and is not part of the line's content.

A **soft space** is exactly one U+0020 SPACE character.

A **hard space** is two or more consecutive U+0020 SPACE characters. A **hard-space run** is a maximal such contiguous sequence; the **position** of a hard-space run is the column of its first character.

A **blank line** is a line containing only U+0020 SPACE characters, or no characters at all.

A **parenthetical symbol** is one of the eight bracket characters: (,), [,], <, >, {, }.

A **phrase** is a maximal contiguous sequence of non-linefeed, non-separator characters on a line, where separators are determined by the phrase-separation rules. A phrase MAY contain soft spaces; see §10.3.

The **beginning** of a non-blank line is the first non-space character on the line.

An **ordinary line** is any non-blank line that is not a comment line (§11.1), a tabulation line (§16.1), or a payload line of a source atom (§14) or literal atom (§15).

6. Root Structure

A parsed TEL document has the following root structure:

```
interface Document {
  directive: string | null;
  pragma: Pragma | null;
  lineEndings: "LF" | "CRLF";
  children: Block[];
}
```

```
interface Pragma {
  version: [number, number];
  schema: string | null;
  sigil: Sigil | null;
}
```

```
type Sigil =
  | "!"
  | "'"
  | "#"
  | "$"
  | "%"
  | "&"
  | ""
  | "*"
  | "+"
  | ","
  | "_"
  | "."
  | "/"
  | ":"
  | ";"
```

```
| "="  
| "?"  
| "@"  
| "\\ "  
| "^"  
| " "  
| "- "  
| "| "  
| "~";
```

7. Interpreter Directive

If the first two characters of the document are `#!` (NUMBER SIGN, EXCLAMATION MARK), then the first line of the document is an interpreter directive line. If not, the interpreter directive is absent.

The interpreter directive payload is the content of the first line after the leading `#!`, up to but excluding the line terminator.

If a document has an interpreter directive and also has a pragma, then the pragma **MUST** appear after the interpreter directive.

An interpreter directive line is not part of the children sequence. It is not part of the semantic content of the document, but should be invariant under reserialization.

8. Pragma

If present, the pragma **MUST** be the first non-blank line after any interpreter directive line, and is parsed as an ordinary line (**E102**).

If present, the entire pragma line **MUST** be fully contained within the first 4096 bytes of the document (**E103**).

The first phrase on the pragma line (its keyword, as defined in §10) **MUST** be `tel`. The keyword `tel` is reserved: it **MUST NOT** appear as a `Field.keyword` or `Variant.keyword` in any `Struct` within a schema (**E209**).

The pragma line **MUST** contain at most three phrases after `tel` (version, schema identifier, and sigil). Any additional phrases are invalid (**E123**). A pattern of the form `<sigil> <text>` that would otherwise be treated as an inline comment does not apply on the pragma line; any such content is invalid (**E123**).

The positional form of the pragma is:

```
tel 1.0 schema-id #
```

The parameters are interpreted in order as follows:

1. TEL version
2. schema identifier
3. sigil

The version parameter **MUST** have the form $x.y$, where x and y are non-negative integers (**E104**). x is the major version and y is the minor version.

The following rules govern how the version number changes across revisions of this specification:

- A revision that rejects a document that would have been accepted by the previous revision **MUST** increment the major version.
- A revision that accepts a previously accepted document but assigns it a different interpretation in its presentation or semantic model **MUST** increment the major version.
- A revision that accepts documents which would not have been accepted by an earlier revision, but does not reject or reinterpret any previously valid document, **MUST** keep the same major version and increment the minor version.

The schema identifier parameter is optional.

The sigil parameter is optional.

The sigil **MUST** be a single ASCII symbolic character. It **MUST NOT** be SPACE, LINEFEED, CARRIAGE RETURN, a letter, a control character, a digit, or a parenthetical symbol (§5) (**E105**).

The default sigil is #, used unless the pragma or the document schema specifies a different one.

8.1 Schema Identifier

The schema identifier, if present, **MUST** be one of:

- an HTTP or HTTPS URL, optionally with a fragment (the # separator and everything after it) that is the **BASE-256-encoded schema signature** of the schema (as defined in §8 of the BinTEL Specification)
- a bare BASE-256-encoded schema signature of the schema

A schema identifier that does not match either of these forms is invalid (**E122**).

The # used in the URL form is the standard URI fragment separator (RFC 3986 §3.5). A bare signature is distinguished from a URL by the absence of a `://` substring. The BASE-256 alphabet (see the BASE-256 Specification §4) consists entirely of Unicode letters and ASCII digits — no whitespace or punctuation — so a schema identifier always occupies a single phrase and is selected as a single word by a double-click in any conforming text-handling environment.

A **schema signature** is a deterministic byte string derived from the SHA-256 hashes of the schema's components (base schema and any layers, in order). It is constructed as a **palimpsest** of those hashes at byte cadence $k = 2$ (see §8 of the BinTEL Specification and the Palimpsest Specification). The palimpsest form is what the pragma carries: it not only identifies the fully composed schema but also encodes the **identities of the base and each layer in order**, so that a receiver holding a library of known schemas and layers can decode the signature to reconstruct the exact composition.

For a non-layered schema (one component), the signature is 32 bytes (32 BASE-256 characters) — exactly the schema’s value hash. For a schema with n total components, the signature is $30 + 2n$ bytes ($30 + 2n$ BASE-256 characters). A producer that wishes to extend a schema with additional layers publishes a new signature by appending each layer’s hash to the palimpsest; a consumer that decodes the signature against its library reconstructs the same composition that the producer intended (§20.3 of this specification).

8.2 Schema Resolution

A schema may be supplied in two independent ways when parsing a TEL document:

- an **invocation schema**, supplied directly to the parser by the calling application
- a **document schema**, identified by the schema identifier in the pragma

The following table defines the outcome for each combination:

Invocation schema	Document schema	Outcome
absent	absent	Untyped document; only the presentation model is available
absent	present	Semantic model available, but types are not statically known
present	absent	Semantic model available with statically known types
present	present, matching	Same as invocation-only; types are statically known
present	present, compatible	Parsed with invocation schema; types are statically known
present	present, incompatible	Error

Types are **statically known** when the schema is available at compile time (or equivalent) in the host language, enabling type-safe access through generated types, type providers, or similar mechanisms. When types are not statically known, the semantic model is still available but must be accessed through a dynamic or generic interface.

Two schema identifiers **match** if:

- both carry a signature, and the signatures are identical; or
- neither carries a signature, and the URLs are identical

A schema identifier that carries a signature takes precedence for matching purposes: a URL-only identifier and a URL-with-signature identifier for the same URL do not automatically match (the signature is authoritative).

A document carrying signature S_{doc} is **compatible** with a consumer carrying signature S_{cons} iff $S_{doc} <: S_{cons}$ under §24 (the formal subtype relation). The signature-subsequence rule is the concrete decision procedure: $S_{doc} <:$

S_cons iff S_cons 's decoded hash sequence is a subsequence of S_doc 's. Every component of S_cons (base schema and layers, in order) appears in S_doc in the same order, but S_doc may include additional layers between or after them.

Compatibility is directional. Under the Liskov Substitution Principle (§24.5), a consumer expecting S_cons MAY read any document whose carried signature S_doc satisfies $S_doc \leq S_cons$, because every constraint imposed by S_cons is also satisfied by S_doc . The converse does not hold in general: a consumer expecting S_doc cannot necessarily read a document carrying a supertype S_cons , since S_doc may require additional layers (and therefore members) that the supertype does not supply.

Resolution Protocol

A parser presented with a document schema MUST resolve it to a Schema value (as defined in §20) before any rule that depends on the schema is applied. Resolution proceeds in the following order:

1. **Built-in lookup.** If the document schema's signature equals the value hash of the built-in `tel-schema` schema (§20.5), the parser MUST use the built-in Schema and skip the remaining steps.
2. **Cache lookup.** A parser MAY maintain an in-memory or on-disk cache keyed by schema signature. If the cache contains a Schema whose composed signature equals the document schema's signature, the parser MUST use that cached Schema.
3. **Library lookup.** If the document schema's signature decodes (per §8 of the BinTEL Specification) against the parser's library of known schemas and layers, the parser MUST use the composition described by the decoded hash sequence.
4. **URL fetch.** If the document schema carries a URL, the parser MAY fetch the body of that URL over HTTPS (or HTTP, where the deployment permits non-confidential carriage). HTTPS MUST be supported by any conforming network-capable implementation; HTTP support is OPTIONAL. The body MUST be a TEL document conforming to the `tel-schema` schema; on parse failure the resolution fails. The parser MAY follow up to a small fixed number of HTTPS redirects (3 is RECOMMENDED); it MUST NOT follow HTTPS-to-HTTP redirects.
5. **Failure.** If none of the above produce a Schema, resolution fails and the parse is aborted. The implementation MUST report a runtime resolution error identifying which step failed.

A non-network-capable parser (for example, one embedded in a build tool with no outbound connectivity) MAY omit step 4; in that case it MUST treat any document schema not satisfied by steps 1–3 as a resolution failure.

Signature Verification

When a document schema's identifier carries a signature (either as a URL fragment or as a bare signature) and a Schema is obtained by URL fetch, the parser MUST verify integrity by:

1. Computing the value hash (§3 of the BinTEL Specification) of the fetched schema document's BinTEL encoding.
2. Composing the value hashes of any layers identified by the signature into a candidate signature per §8 of the BinTEL Specification.

3. Comparing the candidate signature, byte-for-byte, with the signature carried by the document schema.

If the comparison fails, the fetched schema **MUST** be discarded and resolution fails. A parser **MAY** cache only verified schemas.

When the document schema carries a URL with no fragment (no signature), no verification is possible; the parser **MUST** treat the fetched body as authoritative, with the understanding that the binding is then by URL alone.

Caching

Schema bodies **MAY** be cached indefinitely after successful resolution and verification, keyed by signature. URL-only resolutions (no signature) **SHOULD** honour standard HTTP cache headers; an implementation that does not support HTTP caching **MUST** treat URL-only schemas as freshly resolvable on every parse.

Layered-Signature Decomposition

When the document schema's signature contains more than one component ($30 + 2n$ bytes for $n > 1$, per §8 of the BinTEL Specification), the parser **MUST** decompose it against its library of known hashes before parsing the document body. Decomposition produces an ordered sequence $h_0, h_1, \dots, h_{n-1}$ of component value hashes; the parser **MUST** construct the composed Schema by applying the layers identified by $h_1 \dots h_{n-1}$ to the base schema identified by h_0 , in that order, using the merge algorithm of §20.3. If any component hash is unknown to the parser's library and cannot be retrieved by URL (the signature alone does not encode a URL for any component), resolution fails.

Runtime Resolution Error

A resolution failure is a runtime error, not a parsing error: it is reported outside the E1xx / E2xx / E3xx taxonomy. Implementations **SHOULD** report it with sufficient detail for the user to distinguish the failure modes (built-in mismatch, library miss, fetch failure, signature verification failure, malformed body).

8.3 Sigil Resolution

The sigil is determined in the following order of increasing precedence:

1. The default sigil (#)
2. The sigil declared by the resolved schema, if any
3. The sigil specified in the pragma, if present

The sigil **MUST** be determined before parsing any content after the pragma line. If the effective sigil requires the schema (i.e., the pragma does not specify a sigil and the schema may declare one), the parser **MUST** resolve the schema before continuing.

The sigil declared by a schema is given by the `sigil` field of the Schema type (§20).

The pragma line is not included in the `Document.children` sequence. It is recorded only in the `Document.pragma` field.

9. Lines, Margin, and Indentation

A TEL document MAY begin with zero or more blank lines.

A document containing no non-blank lines (other than an interpreter directive or pragma) is valid and has an empty `children` list.

The **margin** is determined as follows:

- If the document begins with an interpreter directive, the margin is zero.
- Otherwise the margin is the sequence of leading spaces on the first non-blank line of the document. The pragma line, if present, is the first non-blank line and therefore sets the margin. (As an ordinary line per §8, the pragma is subject to all line-level rules of this section, including margin determination.)
- If the document contains no non-blank lines, the margin is zero.

Every non-blank line in the document MUST begin with the margin, optionally followed by additional spaces. A non-blank line which does not begin with the margin is invalid (**E106**).

For each non-blank line, the number of spaces following the margin MUST be even (**E107**). The **indent** is defined as one half of the number of spaces between the margin and the first non-space character.

Therefore, after removing the margin, indentation is measured in units of two spaces, and the first non-blank line necessarily has indent 0. Blank lines have no defined indent.

Trailing spaces on a non-blank ordinary line are not permitted (**E108**).

Blank lines have no structural effect, except as explicitly noted in §14 (source atoms), §15 (literal atoms), and §16 (tabulated blocks).

10. Keywords and Inline Atoms

Each non-blank ordinary line is parsed into one or more phrases by the phrase-separation rule (defined in §10.3 below).

The first phrase on a non-blank ordinary line is the **keyword**.

Each subsequent phrase on that line is an **inline atom**.

10.1 Keyword Characters

A keyword may contain any Unicode code point except U+0020 SPACE and U+000A LINE FEED. Non-printing code points are NOT RECOMMENDED in keywords but are not forbidden. Although non-ASCII keywords are permitted, ASCII keywords are generally RECOMMENDED for interoperability and readability.

10.2 Inline Atom Characters

An inline atom may contain any Unicode code point other than LF, subject to the phrase-separation rules defined in §10.3 below.

10.3 Phrase-Separation Rule

After the keyword, the line is parsed left-to-right.

Initially, a single space is sufficient to terminate a phrase and begin the next phrase.

If a hard-space run occurs anywhere on the same line after the keyword, then from the **start** of that run onward, only hard-space runs terminate phrases. Soft spaces after that point become part of the current phrase.

Accordingly:

- before the start of the first hard-space run on a line, either a soft space or a hard-space run terminates a phrase
- from the start of the first hard-space run onward, only a hard-space run terminates a phrase
- from the start of the first hard-space run onward, a soft space becomes content within the current phrase
- each new line resets this rule

Consequently, from the start of the first hard-space run onward, a phrase may contain soft spaces but may not contain hard-space runs.

11. Comments and Remarks

TEL recognises two presentation-only constructs introduced by the sigil but not contributing to the semantic model:

- a **comment**, which occupies an entire line and is represented as a line-level presentation node,
- a **remark**, which is attached to a compound line and is not an ordinary child node.

A third sigil-introduced construct, the **tabulation** line, also exists; tabulations are defined together with the tabulated blocks they introduce in §16.

The document's **sigil** — the character that introduces comments, remarks, and tabulations — is determined by the resolution rules in §8.3.

11.1 Comment

A line is a comment line if, after its leading indentation, its keyword is exactly equal to the sigil, and the line does not qualify as a tabulation line. A line qualifies as a tabulation line if at least one further occurrence of the sigil appears on the line preceded by a hard space; in that case the line is a tabulation line and not a comment line, regardless of any other content.

If the sigil is followed immediately by the end of line, the comment payload is the empty string.

If the sigil is followed by at least one space character, the first such space is consumed as the comment introducer; the comment payload is the remainder of the line, preserved exactly (including any additional leading or internal spaces).

A phrase such as #foo (the sigil concatenated with other characters) is not a comment keyword. This makes it possible to use the sigil as part of a word.

The payload of a comment is not further parsed. Spaces inside the payload are preserved exactly.

Comments participate in indentation and structural ordering as line-level nodes. Comments cannot have children.

A comment line **MUST** be immediately preceded by one of the following: a blank line, another comment line, the start of the document (i.e., a comment may be the very first non-blank line), or a non-blank line at a **strictly lesser indent** than the comment itself — that is, the comment opens a new (deeper) child block of that preceding compound (**E109**). Because a blank line terminates any active tabulated block, this rule ensures that comments cannot appear inside tabulated blocks.

Example:

```
parent          # indent 0
  # comment      # indent 1 — preceded by a non-blank line at
  lesser indent (0); valid
  child          # indent 1
```

A comment is **attached** to the immediately following node if there is no blank line between the comment and that node *and* the following node is at the same indentation level as the comment. The following node may be a compound node, or a tabulation line (in which case the comment is attached to the tabulated block that the tabulation line introduces). A comment that is followed by a blank line, by end of input, by a line at a shallower indentation level, or by a line at a deeper indentation level is **free-standing**.

Comment attachment is a semantic property recorded in the presentation model. It is significant during programmatic editing: when a node is moved or deleted, its attached comments travel with it or are removed with it.

11.2 Remark

A remark is attached to the compound defined by its line.

A remark begins when the sigil appears at the start of a phrase and is immediately followed by exactly one soft space. Whether a given occurrence of the sigil is at the start of a phrase depends on the phrase-separation mode in effect at that point (§10.3).

Accordingly:

- the sigil followed by end of line is an ordinary phrase, not a remark introducer
- the sigil followed by a hard space is an ordinary phrase, not a remark introducer

- the sigil not preceded by a phrase boundary in the current mode is ordinary content within the current phrase
- the sigil at a phrase boundary followed by a soft space introduces a remark

The remark payload begins at the first character after that soft space and continues unchanged to the end of the line.

The sigil itself is not part of the remark payload.

A remark payload is not further parsed.

A compound may have at most one remark.

Remarks do not terminate or split a tabulated block.

12. Compound Tree Structure

Each non-comment non-tabulation ordinary line defines a Compound node whose keyword is the line keyword.

Each subsequent inline atom after the keyword defines an Atom.Inline attached to that compound, unless superseded by the remark rule.

After its inline atoms, a compound may have zero or more child blocks (§17), determined by indentation and blank-line structure.

13. Parent, Child, and Peer Relations

For each non-blank line after the first non-blank line, excluding lines consumed by source atoms or literal atoms, let the **previous compound line** be the most recent preceding non-blank compound line (i.e., excluding comment lines and tabulation lines):

- if its indent is exactly one greater than that of the previous compound line, it is a child of the previous compound line
- if its indent is equal to that of the previous compound line, it is a peer of the previous compound line
- if its indent is less than that of the previous compound line, it closes one or more open compounds and becomes a peer of the nearest preceding compound line with the same indent; if no preceding compound line has the same indent, the document is invalid (**E110**)

A line may not have indent greater than one plus the indent of the previous compound line, except where the source-atom or literal-atom rules apply (**E111**).

Comments and tabulations follow the same indentation and peer/child rules as compounds during parsing, except that comments and tabulations cannot have children. A line that would become a child of a comment or tabulation is invalid (**E112**). In the resulting presentation model, comments and tabulations are absorbed into Block nodes (§17) and do not appear as standalone siblings of compounds.

14. Source Atoms

If a line immediately follows a compound line with no intervening blank line, and its indent is exactly two greater than that compound line's indent, then it begins a source atom, provided:

- the preceding compound does not already have a source atom or literal atom

A source atom is represented in the presentation model as `Atom.Source(text)` and is appended to the end of the atom sequence of the immediately preceding compound.

A compound may have at most one source atom. Introducing a source atom when the preceding compound already has a source or literal atom is invalid (**E113**).

The source atom begins on the double-indented line and includes that line together with each subsequent line until either:

- the end of the document is reached, or
- a non-blank line is encountered whose indent is less than the indent of the first source-atom line

Blank lines are permitted within a source atom.

The captured lines (in order) are converted to a single text string by appending LF after each line, including the last. The array of captured lines therefore yields a text field of the form `line_0 LF line_1 LF ... LF line_{n-1} LF` — every captured line is LF-terminated. A source atom that ends at end-of-file inherits the file's trailing LF as the terminator of its final captured line; a source atom that ends at a dedent likewise inherits the LF of the preceding non-blank line. A blank line within a source atom contributes a zero-length captured line; consecutive blank lines yield consecutive empty captured lines, each contributing one LF.

For each non-blank captured line, exactly the indentation of the first source-atom line is stripped from the start of the line. Any surplus leading spaces are preserved.

For each captured line, trailing spaces are stripped. (Source-atom lines are not ordinary lines, so E108 does not apply to them; trailing spaces are silently removed rather than being an error.)

A blank line within a source atom contributes a zero-length segment, yielding two consecutive LFs in the joined text when the surrounding lines are non-empty.

Line content is otherwise captured literally. In particular, the sigil has no special meaning inside a source atom.

Source-atom lines are subject to the normal line rules (§5): in CRLF mode, the CR preceding each LF is part of the line terminator and is not part of the line content. The LF characters that appear in text are the synthetic separators introduced by the join above; they are not the document's literal line terminators.

Source-atom lines are not compounds and are never members of a tabulated block. A source atom always terminates any surrounding tabulated block.

After a source atom ends, parsing resumes normally. The next non-source-atom line is evaluated for indentation relative to the compound that introduced the source atom, as if the source atom lines were not present.

15. Literal Atoms

If a line immediately follows a compound line with no intervening blank line, and its indent is exactly three greater than that compound line's indent, then it begins a literal atom, provided:

- the preceding compound does not already have a source atom or literal atom

A literal atom is represented in the presentation model as `Atom.Literal(text)` and is appended to the end of the atom sequence of the immediately preceding compound.

A compound may have at most one literal atom. Introducing a literal atom when the preceding compound already has a source or literal atom is invalid (**E114**).

The opening literal-atom line is not part of the payload.

The remainder of that opening line, from its beginning up to but excluding the line terminator, is the delimiter.

The delimiter **MUST** consist only of ASCII characters other than whitespace (spaces, linefeeds, carriage returns, tabs, and other ASCII control characters).

If the delimiter is empty (the candidate opening line contains only its leading indentation and no further content), the line does not begin a literal atom; it is then a blank line per §5 and contributes no structural effect. Indentation on a blank line is not significant.

The literal payload begins immediately after the LF that terminates the delimiter line.

The closing delimiter is identified by scanning for a LF immediately followed by the exact delimiter characters and then immediately followed by another LF. The match is performed against the **raw byte stream of the document**, without any margin stripping or indentation processing: the closing delimiter line therefore begins at column zero of the document, *not* at the opening indent. The scan uses bare LF regardless of the document's line-ending mode. The payload is everything between the opening LF (exclusive) and the closing LF before the delimiter (exclusive). The LF after the closing delimiter terminates the literal atom.

For example, a literal atom nested two indent levels deep with delimiter `END` looks like this (note that the closing `END` is flush left, not indented):

```
outer
  inner
    END
      leading-space-preserved-content
END
  sibling-of-inner
```


Accordingly, an empty literal payload (a LF immediately followed by the delimiter and a LF) is permitted.

The literal payload preserves leading spaces, trailing spaces, internal spaces, and all other content exactly.

If the end of file is reached before a closing delimiter is encountered, the document is invalid (**E115**).

The sigil has no special meaning inside a literal atom.

The line-ending mode rules of §4 do not apply anywhere inside a literal atom payload, nor to the three structural LF characters that bound it (the opening LF, the LF before the closing delimiter, and the LF after the closing delimiter). Every byte between the opening LF and the closing-delimiter LF — including any CR, bare LF, or CR LF sequence — is payload content. Only the bare LF characters that frame a closing-delimiter match are structurally significant, and only for the purpose of identifying that match.

Literal atom payload content is raw: it is not subject to any TEL parsing rules. Indentation, trailing spaces, and all other content are preserved exactly. The only termination condition is a LF immediately followed by the delimiter and another LF.

Literal-atom lines are not compounds and are never members of a tabulated block. A literal atom always terminates any surrounding tabulated block.

After the closing delimiter line and its line terminator, parsing resumes normally. The next non-literal-atom line is evaluated for indentation relative to the compound that introduced the literal atom, as if the literal atom lines were not present.

16. Tabulations and Tabulated Blocks

A **tabulation line** introduces a **tabulated block**: a run of one or more compound lines (called **rows**) sharing a fixed column layout. The tabulation line declares the columns; the rows below fill them.

16.1 Tabulation Line

A line is a tabulation line if, after its leading indentation, its first non-space character is the sigil, and at least one further occurrence of the sigil appears on the line immediately preceded by a hard space.

Each marker occurrence on a tabulation line is identified as follows: the first non-space character (M_0) is always a marker; any subsequent occurrence of the sigil that is immediately preceded by a hard space is also a marker ($M_1, M_2, \dots$).

A tabulation line is represented as a distinct presentation node. Remarks are not applicable to tabulation lines; any content on a tabulation line that would otherwise form a remark is instead part of the heading text for the final column.

The markers on a tabulation line are ordered by position. Let their character offsets from the start of the line, after removing the document margin, be $M_0 < M_1 < \dots < M_n$, where $n \geq 1$. The first marker (at M_0) is the line's keyword and carries no

column semantics. Each subsequent marker defines a column of the tabulated block: marker at M_i (for $i = 1, \dots, n$) defines the start of **column i**. Columns are numbered from 1.

For each non-final column i (where $1 \leq i < n$), its maximum content width is $M_{i+1} - M_i - 2$ code points. The final column ($i = n$) is unbounded.

Column headings. Each marker M_i is followed by a **column heading**, parsed as follows:

- If M_i is immediately followed by end of line, the heading is the empty string.
- If M_i is immediately followed by exactly one space (a soft space), the heading is the text beginning after that space and ending immediately before the first hard space encountered, or at end of line if no hard space follows. If the heading ends at a hard space, the character immediately after that hard space **MUST** be the sigil (i.e., M_{i+1}) (**E120**). The heading text **MUST NOT** itself contain the sigil (**E120**).
- If M_i is immediately followed by two or more spaces (a hard space), the character immediately after those spaces **MUST** be the sigil (i.e., M_{i+1}), and the heading for M_i is the empty string (**E120** if not).
- Any other character immediately following M_i (including a non-space character) is invalid (**E120**).

The column heading for M_0 labels the keyword and pre-column area of rows. The column heading for M_i ($i \geq 1$) labels column i and is positioned within column i 's span on the tabulation line.

Column headings are preserved in the `Tabulation` node as an ordered list parallel to `markerOffsets`. An empty string heading is permitted.

Examples:

- `# ID # Name # Age` — three markers; headings `["ID", "Name", "Age"]`
- `# # Name # Age` — M_0 followed by hard space then M_1 ; headings `["", "Name", "Age"]`
- `# ID # # Age` — M_1 followed by hard space then M_2 ; headings `["ID", "", "Age"]`
- `# foo # # bar` — invalid (**E120**): heading for M_1 would contain the marker
- `# foo # bar # baz` — invalid (**E120**): M_1 followed by hard space not immediately preceding a marker

16.2 Tabulated Block

A **tabulated block** begins immediately after a tabulation line and continues through each subsequent non-blank line until a blank line is encountered or the end of the document is reached. Lines within a tabulated block (other than the tabulation line itself) are called **rows**.

In the presentation model, a tabulated block is represented as a `Block` (§17) whose `tabulation` field holds the tabulation line and whose `compounds` list holds the rows.

A second tabulation line appearing within a continuous run of rows (without an intervening blank line) terminates the current Block and begins a new Block with the new tabulation. The new block's `trailingBlankLines` on the preceding block is zero, indicating that no blank lines separate the two tabulated sub-blocks.

Every non-blank row **MUST** be an ordinary compound line (comments cannot appear inside a tabulated block, since the blank line that would have to precede a comment per §11.1 would itself terminate the block). Every row **MUST** have the same indent as the tabulation line (**E116**). Rows **MUST NOT** have child line-nodes (**E112**).

Row structure. Each row consists of a keyword and zero or more **pre-column atoms**, followed by zero or more **column values**. The keyword and pre-column atoms — that is, the portion of the row before the first hard-space run — are parsed using the same phrase-separation rules as ordinary lines (§10.3). From the first hard-space run onward, the column-based grammar below **replaces** §10.3: column boundaries are derived from the marker offsets M_i declared on the tabulation line, not from phrase mode, and the rules below override the §10.3 phrase classification for that portion of the row.

Spacing constraints. The following two rules govern the spacing on every row:

1. Every contiguous run of two or more space characters (a hard space) **MUST** end at position $M_i - 1$ for some column i that is present on the row (**E117**).
2. No two consecutive space characters may appear at any other position on the row (that is, within the keyword, within pre-column atoms, or within a column value) (**E118**).

These rules together imply:

- the keyword and pre-column atoms are separated from each other by exactly one space
- before each present column i there is exactly one hard space run, ending at $M_i - 1$
- column values contain no internal consecutive spaces

Column presence and values. Column i is **present** on a row if the row contains space characters at both position $M_i - 2$ and position $M_i - 1$ (the mandatory minimum for the hard-space separator). Column i is **absent** from a row if the row ends before reaching position $M_i - 2$; a row need not specify all columns and may omit any suffix of columns.

A present column has an **empty value** if position M_i is itself a space character or the row ends at position $M_i - 1$. An empty value requires that the subsequent column (if any) is also present, since otherwise the separator spaces at $M_i - 2$ and $M_i - 1$ would be trailing spaces, which are not permitted. A row **MUST NOT** have trailing spaces (**E108**).

Omitted column semantics. When a schema is available, an absent column is interpreted according to the schema member that corresponds to that column's position: if the member has a `Scalar` type with a non-null default, the default value is used; if the member is not required, the member is treated as absent (unfilled); if the member is required and has no default, the document is invalid (**E307**).

Width constraint. For each present non-final column i , its value MUST NOT exceed $M_{\{i+1\}} - M_i - 2$ code points in width (**E119**). The final column is unbounded.

Remarks. Remarks are permitted on rows. The hard space that introduces a remark, and the remark payload itself, are exempt from the column spacing constraints and are not subject to column-width limits.

If a row violates any of these constraints, the document is invalid (see **E116** through **E119**).

17. Presentation Nodes

The presentation-layer node types — referenced by name throughout §11–§16 — are:

```
interface Compound {
  keyword: string;
  atoms: Atom[];
  remark: string | null;
  children: Block[];
}
```

```
interface Block {
  comments: Comment[];
  tabulation: Tabulation | null;
  compounds: Compound[];
  trailingBlankLines: number;
}
```

```
interface Comment {
  text: string;
}
```

```
interface Tabulation {
  markerOffsets: number[];
  headings: string[];
}
```

```
type Atom = Inline | Source | Literal;
```

```
interface Inline {
  text: string;
  precedingSpaces: number;
}
```

```
interface Source {
  text: string; // captured lines joined with single LF
                separators
}
```

```
interface Literal {
```

```

    delimiter: string;
    text: string;
}

```

These distinctions are presentation-only.

In the semantic model, all atom presentation forms are just atoms, and the distinction between atom and compound disappears in favor of typed nodes with types.

A **Block** is the primary structural grouping within a compound's children. It consists of, in order:

- zero or more **comments** (contiguous, with no blank lines between them)
- an optional **tabulation** line, which applies to all compounds in the block
- zero or more **compound children** (rows if the tabulation is present, ordinary children otherwise)
- a count of **trailing blank lines**: the number of blank lines that follow the last compound (or, if compounds is empty, the last comment) in the block

A block has at most one tabulation. If a tabulation is present, it **MUST** appear after any attached comments and before the first compound child. A block with a tabulation **MUST** have at least one compound child (row).

A block whose compounds list is empty and whose comments list is non-empty represents a free-standing comment group (a comment or comments not immediately followed by any compound at the same level). Such a block has no tabulation.

Each `Atom.Inline` records the number of spaces immediately preceding it on its source line. Each `Atom.Literal` records the delimiter string used to open and close it, in addition to the payload text.

18. Presentation Model and Semantic Model

TEL defines both a presentation model and a semantic model.

18.1 Presentation Model

The presentation model is constructed during parsing. When a schema is available, parsing and type assignment proceed together — the result is a presentation model and a semantic model produced in a single pass — and the schema is consulted to disambiguate odd-indented lines (the schema-aware **E107 recovery** of §19.5). When no schema is available, the parser falls back to the schema-independent **shallower-wins** rule on E107; the rest of the recovery table of §19.5 is schema-independent in either case.

It preserves:

- the optional interpreter directive
- the optional pragma
- each compound's keyword, atoms, remark, and child blocks

- each block's attached comments, optional tabulation, ordered compounds, and trailing blank-line count
- atom presentation form (`Inline`, `Source`, or `Literal`)
- for each inline atom, the number of spaces immediately preceding it
- ordering and structure derived from indentation

A conforming serializer **MUST** produce output that, when parsed, yields an identical presentation model. Specifically, the serialized output **MUST** preserve:

- all compounds, with their keywords, atoms, remarks, and children
- all blocks, with their comments, tabulations (including marker offsets and headings), compounds, and `trailingBlankLines` counts
- for each `Inline` atom, the `precedingSpaces` count
- document-level fields: interpreter directive, pragma, and children order

A serializer **MAY** apply the following normalizations, since the affected details are not recorded in the presentation model:

- Blank line content **MAY** be normalized to empty lines (rather than space-only lines).
- A minimum hard space (exactly two spaces) **MAY** be used before remark introducers.
- Multiple consecutive trailing blank lines at the end of a block **MAY** be collapsed to the recorded `trailingBlankLines` count.

All other presentation-model details **MUST** be reproduced exactly. In particular, the round-trip guarantee preserves: all compounds with their keywords, atoms, remarks, and children; all block structure including comments, tabulations, and ordering; atom presentation form and `precedingSpaces`; and the `Literal.delimiter` string.

18.2 Semantic Model

The semantic model is derived from the presentation model by applying the type assignment algorithm (§20.2) during parsing. The result is a tree of `Element` values:

```
type Element = Node | Value;
```

```
interface Node {
  keywordIndex: number | null; // null only for the document root
  type: Type;
  children: Element[];
}
```

```
interface Value {
  keywordIndex: number;
  type: Scalar;
  text: string;
}
```

A `Node` represents a `Struct`-typed or `Flag`-typed element. `Node.type` is the `Type` assigned by the type assignment algorithm (§20.2). `Node.children` is the ordered list of child elements; for a `Flag`-typed node, `children` is always empty.

A `Value` represents a `Scalar`-typed element. It is a leaf: it carries the atom text in `Value.text` and has no children.

Every non-root element carries a `keywordIndex`: the position of the element's keyword in the keyword order of the parent's `Struct` type (§20). The document root has `keywordIndex = null`. The `keywordIndex` identifies which member (and, for `Select` members, which variant) the element fills, and is sufficient to recover the keyword string from the schema.

The interpreter directive, `pragma`, comments, tabulations, and remarks are not part of the semantic model. There is a one-to-one mapping between presentation-layer atoms and compounds on the one hand, and elements on the other: every atom and every compound maps to exactly one element.

18.3 Mapping Procedure

The mapping from presentation model to semantic model proceeds as follows. The type assignment algorithm (§20.2) ascribes a type to every atom and compound. Given these assignments, the semantic tree is constructed by:

1. **Document root.** Create a root `Node` with type equal to `Schema.document` and children constructed from steps 2–5.
2. **Compound children.** For each compound child `C` of the current node (iterating across all blocks in order, skipping comments and tabulations):
 - If the type assigned to `C` is `Struct` or `Flag`, create a `Node` with that type and children constructed by recursing into `C`'s children (empty for `Flag`).
 - If the type assigned to `C` is `Scalar`, create a `Value` with that type and text equal to the compound's inline atom text if present, or the empty string if the compound has no inline atoms.
3. **Atoms.** For each atom `A` assigned to the current node (in order):
 - If the assigned type is `Scalar`, create a `Value` with that type and text equal to `A`'s text.
 - If the assigned type is `Flag`, create a `Node` with that type and an empty children list.
4. **Ordering.** Atom-derived nodes and compound-derived nodes for the same member are interleaved in the order they were assigned. Atom-derived nodes for a member precede compound-derived nodes for the same member (atoms appear on the parent line, before any child lines).
5. **Defaults.** For each required `Field` member with a `Scalar` type and a non-null default that was not filled by any atom or compound child: create a `Value` with that `Scalar` type and text equal to `Scalar.default`. This node is placed at the position in the child list corresponding to the member's position in member order.

The resulting tree is fully determined by the presentation model and the schema. No ambiguity remains: the type, value, and child order of every element are fixed by the type assignment algorithm (§20.2).

19. Schema-Governed Structure and Error Diagnosis

In addition to parsing errors, a TEL document may be structurally invalid with respect to a schema.

When a schema is available, it is applied during parsing rather than as a separate post-processing stage. This is necessary because the schema is consulted to disambiguate odd-indented lines (the schema-aware **E107 recovery** of §19.5): for a line whose relative indentation is odd, the parser picks the candidate depth at which the line's keyword is a valid member of the parent struct. The result is a presentation model and a semantic model constructed together in a single pass. When no schema is available, indentation recovery falls back to the schema-independent shallower-wins rule of §19.5.

19.1 Atom and Compound Interchangeability

Every presentation-layer atom and every presentation-layer compound corresponds to an element, and every element has a type. The distinction between atom and compound is therefore presentational rather than semantic.

A child whose type can be uniquely inferred from schema position may be written either as an atom in atom position or as a compound with an explicit keyword. A child whose type cannot be uniquely inferred must be written as a compound with an explicit keyword.

Conversely, when a child is written as a compound with an explicit keyword, an implementation may determine that the same child could have been written positionally as an atom, provided the schema would have assigned the same type deterministically.

19.2 Positional Assignment

For a given parent type, the schema defines an ordered sequence of child specifications.

The order in which child types are specified in the schema determines the order in which inline atoms are assigned types.

Inline atoms may be assigned types from that ordered sequence so long as the applicable child specifications are non-repeatable.

If an atom position is assigned to a repeatable child type, then all subsequent inline atoms on that same compound line must be assigned to that same repeatable child type. Consequently, a repeatable `Scalar` member may only consume atoms if it is the last atom-assignable member in member order; no further atoms may be assigned to subsequent members.

Similarly, once atoms are assigned to an `all-Flag Select` member, no further atoms may be assigned to subsequent members, because each atom is matched against the `Select`'s variant keywords and the atom phase cannot advance past a `Select` member except by skipping it entirely.

For a repeatable member, occurrences may be split across both of the following:

- inline atoms on the parent compound line, and
- subsequent compound children of the parent with the same keyword

These two assignment mechanisms may be combined freely. The E309 contiguity rule already prohibits differently-typed compound children from being interleaved between such occurrences. Remark lines do not affect this rule.

19.3 Error Taxonomy

Errors are identified by a code of the form **E1xx** (parsing), **E2xx** (schema), or **E3xx** (validation). Parsing errors indicate violations of the presentation-model syntax. Schema errors indicate a malformed schema. Validation errors indicate that a document does not conform to its schema.

Each error is referenced by code at the point in the specification where its trigger condition is defined.

Diagnostic spans. Every diagnostic MUST identify the relevant region of the document as a half-open span [*start*, *end*) of zero-based code-point offsets from the start of the document. A zero-width span (where *start* = *end*) denotes a point location. The span for each error code is specified in the tables below.

Parsing Errors (E1xx)

Code	Section	Description	Span
E101	§4	BOM present at start of document	The BOM bytes ([0, 3) for a UTF-8 BOM)
E102	§8	Pragma is not the first non-blank line after any interpreter directive	The <code>tel</code> keyword on the misplaced line
E103	§8	Pragma line extends beyond the first 4096 bytes	The entire pragma line
E104	§8	Pragma version parameter does not have the form <code>x.y</code> with non-negative integers	The version atom
E105	§8	Pragma sigil is a space, LF, CR, letter, digit, or parenthetical symbol	The sigil atom
E106	§9	Non-blank line begins with fewer than the margin number of spaces	The leading spaces of the line (zero-width at line start if no spaces)
E107	§9	Relative indentation after the margin is odd	The leading spaces of the line; extended through subsequent lines if margin adjustment persists (see Indentation Recovery below)
E108	§9, §16		The trailing space characters

Code	Section	Description	Span
		Trailing spaces on a non-blank ordinary line or tabulated row	
E109	§11.1	Comment line not preceded by a blank line, another comment, start of document, or lesser-indented line	Zero-width span at the start of the comment line
E110	§13	Line indent does not match any open compound's indent. Reserved: not reachable by the recursive parser of this specification (see §19.5); raised only by implementations that track an explicit ancestor stack.	The leading spaces of the line
E111	§13	Line indent exceeds the preceding non-blank line's indent by more than one	The leading spaces of the line
E112	§13, §16	Line would become a child of a comment, tabulation, or tabulated row	Zero-width span at the start of the line
E113	§14	Source atom introduced when the preceding compound already has a source or literal atom	The first line of the duplicate source atom
E114	§15	Literal atom introduced when the preceding compound already has a source or literal atom	The opening delimiter line of the duplicate literal atom
E115	§15	Literal atom reaches end of file before its closing delimiter line	The opening delimiter line
E116	§16	Tabulated row has an indent different from the tabulation line	The leading spaces of the row
E117	§16	Hard space on a tabulated row does not end at a column start boundary	The misaligned hard-space run
E118	§16	Consecutive spaces appear within a keyword, pre-column atom, or column value on a tabulated row	The consecutive space characters within the value
E119	§16	Column value exceeds the maximum width for that column	The overflowing column value
E120	§16.1	Malformed tabulation line heading	The malformed heading region (from the marker to the next marker or end of line)

Code	Section	Description	Span
E121	§4	CR not immediately followed by LF, or line-ending mode inconsistency	The CR character (or CR LF pair that violates the established mode)
E122	§8.1	Schema identifier is not a valid URL or bare BASE-256-encoded schema signature	The schema identifier atom
E123	§8	Pragma line has extra atoms beyond the expected parameters, or contains a remark	The first extra atom, or the remark introducer

Schema errors (E2xx) and validation errors (E3xx) arise from violations of the schema language rules (§20) and document conformance constraints (§21). Their trigger conditions and diagnostic spans are catalogued at the ends of §20.1 (Schema Validity Constraints) and §21 respectively.

19.4 Error Diagnosis

Error diagnosis in TEL has three layers:

- **parsing diagnosis**, which reports violations of the presentation syntax defined by this specification
- **schema diagnosis**, which reports violations that arise when the presentation model is checked against a schema and translated into the semantic model
- **validation diagnosis**, which reports violations of constraints in the parsing of elements

These three layers SHOULD be distinguished in diagnostics.

A conforming implementation SHOULD report multiple independent errors when validating a document, rather than halting at the first error encountered. Each error has a defined recovery strategy (§19.5) that allows parsing to continue and subsequent errors to be reported.

Every diagnostic MUST include the error code and the span defined for that error in §19.3. The span is expressed as a half-open range [start, end) of zero-based code-point offsets from the start of the document.

19.5 Error Recovery

For every error condition, a conforming implementation MUST apply the recovery strategy defined here before continuing. No error SHALL prevent subsequent errors from being reported.

Parsing Error Recovery

Code	Recovery strategy
E101	Ignore the BOM and continue parsing from the next byte.

Code	Recovery strategy
E102	Restart parsing the entire document using the version, schema identifier, and sigil extracted from the misplaced pragma.
E103	Allow the pragma line to exceed the 4096-byte limit and continue parsing its content normally.
E104	If the version parameter cannot be parsed as x.y at all, ignore it and parse with the latest known version. If it has the correct format but names an unknown version, use the most recent minor version of the same major version if one is known; if the major version itself is unknown, use the latest known version overall.
E105	Ignore the invalid sigil and use the default sigil (#) instead.
E106	If the line has exactly one fewer leading space than the current margin, insert a synthetic leading space and parse the line at the current indentation level normally. If the line has two or more fewer leading spaces than the current margin, reset the margin to the line's actual indentation level from that point forward.
E107	Parse the line's keyword; check which of the two candidate indent levels (± 1 space) makes the keyword valid according to the schema; adjust the margin accordingly. See indentation recovery algorithm below.
E108	Ignore trailing spaces and parse the remainder of the line normally.
E109	Ignore the missing preceding blank line (or other required predecessor) and treat the comment as normally attached to the following node.
E110	Same indentation recovery as E107.
E111	The over-indented line is skipped (omitted from the presentation model) and parsing continues with the following line at the original expected indent.
E112	Same indentation recovery as E107: the line cannot be a child of its apparent parent (a comment, tabulation, or row), so treat it as if indented one level less and use the schema to validate the adjusted placement.
E113	Ignore the duplicate source atom; use the first one encountered.
E114	Ignore the duplicate literal atom; use the first one encountered.
E115	Treat the unclosed literal atom's payload as everything from the opening delimiter line to the end of file (excluding the final LF, if any).
E116	Interpret the tabulated row according to its actual hard-space positions regardless of alignment with column markers. Suppress any further alignment errors (E117, E118, E119) on the same row.
E117	Same as E116.
E118	Same as E116.
E119	Same as E116.
E120	Report the error and continue parsing, but disable column-alignment checking for the remainder of the current tabulated block.
E121	Treat any malformed sequence of consecutive CR and LF characters as a single line break if it contains at most one CR and at most one LF; treat it as two line breaks if either CR or LF appears more than once in the sequence.
E122	Ignore the invalid schema identifier and continue parsing as if no schema identifier were specified. The document is treated as untyped.

Code	Recovery strategy
E123	Ignore the extra atoms and any remark on the pragma line. Parse the pragma using only the first three atoms (version, schema identifier, sigil).

Validation Error Recovery

All validation errors (E301 through E311) are self-contained: they do not have cascading effects on the remainder of the type assignment or validation process. An implementation **MUST** record the error and continue processing remaining nodes as if the erroneous node were absent or were assigned the most plausible available type. Specific recovery notes:

- **E311** (Flag compound with atoms or children): ignore the atoms and children of the Flag compound; treat it as a bare keyword with no content.

Indentation Recovery (E107, E111)

When a line's relative indentation after the margin is odd (E107), the line sits between two valid indentation positions: one space shallower and one space deeper. v1.0 specifies two recovery rules, selected by whether a schema is available at parse time.

Schema-aware recovery (when a schema is available). Let s be the number of spaces after the margin. Let $\text{shallower} = \lfloor s / 2 \rfloor$ and $\text{deeper} = \text{shallower} + 1$. The two candidate depths correspond to two candidate parent structs: at shallower, the parent is the most recent compound (or the document root if $\text{shallower} = 0$) that would accommodate a peer at that depth; at deeper, the parent is the compound at depth shallower (the most recent open compound at that depth).

For each candidate, the parser computes a **validity bit** for the line's keyword K :

- A candidate is **valid** if the parent compound exists, its resolved type (after Reference resolution per §20.2) is a Struct, and K appears in that Struct's keyword order (Field keywords plus the variant keywords of any SelectRef member, per §20).
- A candidate is **invalid** otherwise. This subsumes the cases where the parent doesn't exist (e.g. the line is at the very start of the document and the deeper candidate would require an open compound that isn't there); the parent is a comment, tabulation, or row (which cannot have children, by §13 and §16); the parent's resolved type is Scalar or Flag (also cannot have children); or K is not declared at that position.

The recovery then proceeds:

1. Record an E107 error whose span covers the line's leading whitespace.
2. Choose the placement:
 - if only shallower is valid: place the line at shallower;
 - if only deeper is valid: place the line at deeper;
 - if both are valid OR both are invalid: place the line at shallower (shallower-wins tiebreak).
3. Parsing continues from the next line at the chosen depth.

This procedure is deterministic: a given (line, ancestor-stack, composed schema) triple always produces the same placement.

Schema-independent recovery (when no schema is available). When the parser has no schema (an untyped document, per the absent-invocation-and-document-schema row of §8.2), the schema-aware validity check cannot be performed. The parser falls back to the **shallower-wins rule**:

1. Record an E107 error whose span covers the line's leading whitespace.
2. Treat the line as if its relative indentation were $\lfloor \text{spaces} / 2 \rfloor$ (integer division), i.e. the shallower of the two adjacent even levels.
3. Parsing continues from the next line as if the recovery had not occurred.

The schema-independent rule is also the fallback embedded in step 2 of the schema-aware rule: when both candidates are invalid (or both valid), the line is placed at shallower. A parser that prefers a single code path may implement only the schema-independent rule; the schema-aware rule is then a strict refinement that picks deeper only when shallower would necessarily mis-place the line (the keyword is not declared at shallower's parent but IS declared at deeper's parent).

E111 over-indentation. When a line is indented more than one level deeper than the previous compound line, the over-indented line is recorded as an **E111** error and omitted from the presentation model. Parsing continues with the next line at the originally expected indent. This keeps the parser deterministic without requiring schema-aware indent inference.

E110. §13 also defines **E110** for the case of a dedent that cannot find a matching ancestor indent. The recursive parsing algorithm used here cannot produce that case (every dedent encountered during the recursive descent terminates a `parse_blocks` invocation at the matching ancestor level). E110 is therefore retained in the error catalogue as a reserved code for non-recursive implementations — for example, ones that track an explicit ancestor stack and match dedents against it. Such an implementation surfaces E110 in place of the recursive parser's silent terminations; the recovery is then the same as E107 (treat the line at the shallower of the two candidate indent levels). Under the canonical recursive parser of this specification, E110 never fires.

20. Schema Language

A schema is expressed using the following types:

```
interface Schema {  
  name: string;  
  document: Struct;  
  layers: Layer[];  
  sigil: Sigil | null;  
  records: RecordDefinition[];  
  scalars: ScalarDefinition[];  
  selects: SelectDefinition[];  
}
```

```
interface Layer {  
  name: string;
```

```

    overlay: Struct;
    records: RecordDefinition[];
    scalars: ScalarDefinition[];
    selects: SelectDefinition[];
}

```

```

type Definition =
  | RecordDefinition
  | ScalarDefinition
  | SelectDefinition;

```

```

interface RecordDefinition {
  name: TypeName;
  members: Member[];
  validators: string[];
}

```

```

interface ScalarDefinition {
  name: TypeName;
  validators: string[];
}

```

```

interface SelectDefinition {
  name: TypeName;
  variants: Variant[];
  validators: string[];
}

```

```

type Type = Struct | Scalar | Flag | Reference;

```

```

interface Struct {
  members: Member[];
  validators: string[];
}

```

```

interface Scalar {
  validators: string[];
}

```

```

interface Flag {}

```

```

interface Reference {
  name: TypeName;
}

```

```

type Member = Field | SelectRef | Exclude;

```

```

// Per-axis declaration state. "default" means no flag was
  declared on this
// axis (its effective value is the schema-language default –
  required=true,
// repeatable=false). "loose" means the loosening flag was
  declared on the

```

```

// base side (`optional` or `repeatable`). "tight" means the
// tightening flag
// was declared on the layer side (`required` or `irrepeatable`).
//
// Effective booleans are derived from polarity:
//   required   = (member.required   != "loose")
//   repeatable = (member.repeatable == "loose")
//
// The tristate is retained (rather than collapsing to booleans
// during schema
// construction) so that the layer-merge of §20.3 can detect
// loosening of an
// already-tight axis (E215, E216) regardless of whether the
// layer's declared
// flag agrees with or contradicts the base's effective value.
type Polarity = "default" | "loose" | "tight";

interface Field {
  required: Polarity;
  repeatable: Polarity;
  keyword: string;
  type: Type;
  default: string | null;
}

// References a SelectDefinition at a member position. The named
// Select's
// variants become the keywords admissible at this position; the
// SelectRef
// itself has no own keyword. Polarity lives on the use site
// (here), not
// on the SelectDefinition.
interface SelectRef {
  required: Polarity;
  repeatable: Polarity;
  reference: TypeName;
}

interface Variant {
  keyword: string;
  type: Type;
}

// Layer-only operation. At the position of a Struct, `Exclude(K)`
// would
// have applied at a use-site; in this revision exclusion is
// performed only
// at the SelectDefinition level, where K names a variant of the
// SelectDefinition being merged. The Member-kind form is retained
// for
// symmetry with §20.3 merge tables and is permitted only inside a
// layer's
// SelectDefinition body (not in a Struct), where it removes a
// variant from

```



```

// the merged SelectDefinition. Use outside that position is
// **E217**.
interface Exclude {
  keyword: string;
}

// A TypeName is a PascalCase identifier (§20.7) naming a
// RecordDefinition,
// ScalarDefinition, SelectDefinition, or a built-in type.
// Distinct from a
// kebab-case `identifier` (§20.7), which is used for non-type
// names
// (schema name, layer name, field keywords, variant keywords,
// validator
// names).
type TypeName = string;

```

`Schema.name` is a kebab-case identifier (§20.7) for the schema. It is a human-readable label used to identify the schema in source form; it is **not** the same as the schema identifier carried in a document's pragma (§8.1), which is either a URL or a SHA-256 content hash of the schema's BinTEL.

`Schema.document` is the root Struct that defines the type of the document root compound. It is Struct-typed directly (not Type-typed) by analogy with `Layer.overlay`: every schema must define a root struct, and no other Type variant is meaningful at the document root.

`Schema.layers` is an ordered list of Layer values defining optional schema extensions. The empty list is the normal case for a schema with no layers. Layer composition is defined in §20.3.

`Schema.sigil` is the default sigil for documents that use this schema, or null if the schema does not declare one. When non-null, it MUST satisfy the same character constraints as a pragma sigil (§8): it MUST NOT be a space, LF, CR, a letter, a control character, a digit, or a parenthetical symbol (§5) (**E208**). When a document's pragma omits a sigil but provides a schema identifier that resolves to a schema with a non-null sigil, the schema's sigil is used as if it had been specified in the pragma (§8.3).

`Schema.records`, `Schema.scalars`, and `Schema.selects` are ordered lists of RecordDefinition, ScalarDefinition, and SelectDefinition declarations respectively. Each binds a PascalCase TypeName (§20.7) to a Struct, Scalar, or Sum (union of variants). Together the three lists populate the schema's **Definition namespace** — the union of TypeNames addressable by a Reference or by a SelectRef. A Reference resolves to whichever Definition carries the matching name; structurally, records produce Structs with members, scalars produce Scalars with validators, and selects produce sums with variants. The three lists are kept distinct in the data model because their bodies differ, but they share a **single namespace**: no two Definitions across the three lists may share a name within the composed schema (**E211**).

The Definition mechanism is what makes recursive schemas finitely expressible: the schema-of-schemas itself (see the tel-schema subsection below) is necessarily recursive, and so is any schema whose data has cyclical structure. The empty list is the normal case for non-recursive schemas.

A Layer's name is a kebab-case identifier (§20.7) labelling the layer. It MUST be unique across all layers of a schema (**E205**). `Layer.overlay` is a Struct whose members are merged into the composed schema's root struct by the algorithm in §20.3. `Layer.records`, `Layer.scalars`, and `Layer.selects` are ordered lists of Definitions introduced by the layer; together they merge with the base schema's `Schema.records` $\cup$ `Schema.scalars` $\cup$ `Schema.selects` and any preceding layers' Definitions to form a single namespace visible to all references in the composed schema. The empty lists are the normal case for layers that only extend the root struct.

A RecordDefinition has a name, a list of members, and a list of struct-level validators. The name MUST be a TypeName (§20.7), unique across the composed namespace `Schema.records` $\cup$ `Schema.scalars` $\cup$ `Schema.selects` $\cup$ `Layer.{records, scalars, selects}` (**E211**, with same-name records, scalars, or selects in *layers* triggering Definition merge per §20.3 rather than E211). The members field is a list of Members, structurally identical to those of a Struct: a RecordDefinition is, in effect, a named Struct. `RecordDefinition.validators` mirrors `Struct.validators` (§21.6) and applies to every instance of the Definition.

A ScalarDefinition has a name (subject to the same uniqueness rule above) and a list of validators; it is a named Scalar. Layer-merge of same-name ScalarDefinitions concatenates the validators lists in declaration order, deduplicated.

A SelectDefinition has a name (subject to the same uniqueness rule), a non-empty list of variants, and a list of validators. It is a named sum type. Each variant has a kebab-case keyword (the source-level keyword by which an instance is written) and a type; the type may be any Type (Struct, Scalar, Flag, or Reference). A SelectDefinition's validators list mirrors `Struct.validators` for symmetry: each named validator inspects the chosen variant of an instance and may reject it under cross-cutting rules. Layer-merge of same-name SelectDefinitions removes variants (via `Exclude` members declared in the layer's SelectDefinition body) and appends validators; a layer MUST NOT introduce a variant with a keyword absent from the base SelectDefinition (**E214**).

Flag types cannot be aliased through Definition; they are referenced by the built-in name `Flag` instead (§20.5).

A Reference is a Type whose semantic content is delegated to the Definition it points at by name. During type assignment (§20.2) a value position whose schema type is `Reference(N)` is treated as if its type were the resolved Definition: a RecordDefinition resolves to the Struct formed from its members and validators; a ScalarDefinition resolves to the Scalar formed from its validators. A SelectDefinition is never the resolved type of a plain Reference: a sum at a member position is always introduced via `SelectRef` rather than `Field.type`, because a Field has a single keyword whereas a sum has none.

A Reference whose name resolves to a `SelectDefinition` is invalid (**E218**); a Reference whose name does not match any `Definition` in the composed schema is also invalid (**E210**).

A `SelectRef` is a `Member` kind that places a sum at a member position. Its reference is the `TypeName` of a `SelectDefinition`; the `SelectRef`'s polarity (required, repeatable) lives at the use site. A `SelectRef` whose reference does not match any `SelectDefinition.name` in the composed schema is invalid (**E210**); one that resolves to a non-`SelectDefinition` (a record or scalar) is also invalid (**E218**).

TEL schemas are themselves representable as TEL documents. The TEL schema that describes the TEL schema language is therefore self-describing; the schema for schemas has `Schema.name = tel-schema`. The serialization of a schema as a TEL document is governed by that schema. Because schemas are TEL documents, they have a deterministic BinTEL encoding (see the [BinTEL Specification](#)), which is used for schema hashing and identification (§8.1). The concrete TEL representation of the type model defined above — the keyword vocabulary, member ordering, and validators used to write a schema as a TEL document — is given in §20.6 and embodied in the file `tel-schema.tel`.

A `Struct` has an ordered list of `Members`. Each member describes one logical child slot of the struct and is either a `Field` or a `SelectRef`. Both carry the per-axis polarities `required`: `Polarity` and `repeatable`: `Polarity`. Their effective boolean values are derived:

- `effective required = (member.required != "loose")` — the slot **MUST** appear at least once unless `optional` was declared
- `effective repeatable = (member.repeatable == "loose")` — the slot **MAY** appear more than once only when `repeatable` was declared

Surface syntax. A schema document expresses these two polarities via four `Flag` fields in the `Field` and `Select` records (§20.5). The default for any `Field` or `SelectRef` — neither flag declared — is `required: "default"`, `repeatable: "default"`, i.e. the **tight** cardinality (exactly one). To loosen, a base schema declares one of:

- `optional` — sets `required: "loose"` (effective `required` becomes false)
- `repeatable` — sets `repeatable: "loose"` (effective `repeatable` becomes true)

To re-tighten a member that an earlier layer declared loose, a later layer declares one of:

- `required` — sets `required: "tight"` (overrides a previous `optional`)
- `irrepeatable` — sets `repeatable: "tight"` (overrides a previous `repeatable`)

`required` and `irrepeatable` may also appear in a base schema; there they are redundant no-ops that re-state the default, since the default is already tight on both axes. They are not errors.

The §20.3 merge rule for each axis is: a layer declaration of `"loose"` (i.e. `optional` or `repeatable`) against a merged base whose polarity is `"default"` or `"tight"` is **E215** / **E216** respectively. A layer declaration of `"tight"` against a base of any

polarity is permitted (override). When neither flag is declared on the layer for an axis (`layer.required = "default"`), the base's polarity carries through unchanged. The retention of the tristate, in preference to a simple boolean, is what lets the merge distinguish a layer redundantly restating the existing state from a layer attempting to flip it.

Tightening is subtype-producing (§24); loosening is not.

A `Field` member has a single keyword and a single type. It represents a child whose keyword and type are fixed.

A `SelectRef` member references a named `SelectDefinition`. After Reference resolution, the referenced `SelectDefinition`'s `variants` become the keywords admissible at this position; the `SelectRef` itself has no own keyword. A `SelectRef` value at a member position looks and behaves exactly like one of the referenced sum's variants: if the chosen variant has `Struct` type, the compound child has that struct's members as children; if the variant has `Scalar` type, the compound child carries a value; if the variant has `Flag` type, the compound child is a bare keyword with no content.

`Variant.keyword` is the kebab-case keyword by which a child compound of that variant is written in TEL when explicit. `Variant.type` may be any `Type`.

A member is **atom-assignable** when it can be satisfied by an inline atom rather than only by a compound child:

- A `Field` is atom-assignable iff its type, after Reference resolution (§20.2), is `Scalar` or `Flag`.
- A `SelectRef` is atom-assignable iff every variant of the referenced `SelectDefinition`, after Reference resolution, has `Flag` type.

A member with at least one non-atom-assignable position — a `Field` of `Struct` type, or a `SelectRef` whose referenced sum has any non-`Flag` variant — can only be filled by a compound child with an explicit keyword. This definition is used by the type-assignment algorithm (§20.2) and by the construct operation (§22.2); both refer to it rather than restating it.

A `Scalar` type represents a leaf value constrained by zero or more validators. `Scalar.validators` is an ordered list of helper-method names; each is invoked on the value text during validation, and the value is accepted only when every validator returns `Valid` (AND-conjunction). An empty `validators` list means the `Scalar` accepts any text. Validator semantics are defined in §21.

`Scalar.default` is either `null` (no default) or a string giving the value to be used when the member is absent. A non-null default MAY only be specified if the `Scalar` appears in a `required: true` member; specifying a non-null default on a non-required member is a schema error (**E204**). When a required `Field` member whose type is a `Scalar` with a non-null default is absent from the document, the default value is used as the semantic value and no E307 error is raised.

A `Struct` type MAY likewise carry a `validators` list. A struct validator inspects the entire struct element (its members' values), enabling cross-field constraints such as “postcode is required when country is UK”. Multiple struct validators apply in AND-conjunction. Struct validators are invoked AFTER all of the struct's children have been individually validated. Validator semantics are defined in §21.

Validators on scalars and validators on structs share a single namespace (§21.1): a validator name resolves to one helper method, which dispatches on the request kind at invocation time.

Serialising `Scalar.default`. When a schema is written as a TEL document, the `default` field of a `Scalar` carries an arbitrary string and is serialised using the atom-form escalation algorithm of §22.3: an inline atom is used when the value contains no LF and no hard spaces in soft-space mode; a source atom is used for multi-line values that have no trailing spaces and require no byte-exact preservation; and a literal atom is used otherwise. This mirrors how any `Scalar` value is serialised at a use site, so a schema author writes a `default` exactly as they would write any other scalar value.

A `Flag` type carries no value of its own. Its identity is entirely determined by its keyword (`Field.keyword` or `Variant.keyword`): in compound position, a `Flag`-typed node is written as the keyword alone, with no inline atoms; in atom position, the atom text is matched against the keyword. A `Flag`-typed member SHOULD NOT be required, since a required `Flag` member would be unconditional boilerplate.

Member ordering recommendation. The order of members in a `Struct` determines which children can be serialized as inline atoms (see the `construct` operation in §22.2). To maximize the use of inline atoms, schema authors SHOULD order members as follows:

1. Required `Field` members with `Scalar` type — especially any “identifying” field such as a keyword or name, since placing it first lets the whole field be declared inline with the identifier as the first atom (e.g. `field some-keyword required repeatable`).
2. Non-required `Field` members with `Scalar` type, prioritizing those most likely to be specified rather than absent. Note (§20.8): only the first non-required `Scalar` member is fillable as an inline atom; any subsequent non-required `Scalar` members will require explicit compound children.
3. Non-required `Field` members with `Flag` type. Each such member can be set inline by writing its keyword as an atom; if absent from the atoms, the skip rule in §20.2 advances past it.
4. Either an all-`Flag Select` member or a single repeatable `Field` member with `Scalar` type — but not both, since only one of these can appear in the trailing atom position.
5. All remaining members (`Struct`-typed fields, mixed-variant `Select` members, and any further members), which will always be serialized as compound children.

This ordering is a recommendation, not a requirement. Any member order is valid.

Member order. The **member order** of a Struct is the sequence of its members in their declaration order within `Struct.members`. Where a specification rule refers to members “in member order”, it means iterating members `[0]`, `members[1]`, ..., `members[n-1]`.

Keyword order. The **keyword order** of a Struct is a flat sequence of (keyword, type) pairs obtained by expanding each member in member order: a `Field` member contributes a single entry (its keyword and type); a `SelectRef` member contributes one entry per variant of its referenced `SelectDefinition`, in the declaration order of that `SelectDefinition`’s variants. Keywords are numbered from 0 in this sequence; the position of a keyword in keyword order is its **keyword index**.

Identifier naming convention. Programmatic identifiers defined by this specification — including helper method names in `Scalar.validators` and `Struct.validators` and the edit operation identifiers of §22.2 — use **kebab-case**: a sequence of lowercase ASCII words separated by hyphens (e.g. `update-value`, `attach-remark`). Schemas SHOULD use kebab-case for validator names.

Every kebab-case identifier corresponds to a unique sequence of lowercase words. Implementations SHOULD represent these identifiers idiomatically in their host language by applying the equivalent convention:

- **kebab-case** (`update-value`) — the canonical form used in schemas and in this specification
- **snake_case** (`update_value`) — e.g. Rust, Python
- **camelCase** (`updateValue`) — e.g. Java, TypeScript, JavaScript
- **PascalCase** (`UpdateValue`) — e.g. C#, Go exported names

The mapping between these conventions is an isomorphism over sequences of lowercase words: implementors SHOULD expect identifiers to appear in the idiomatic style of the host language and MUST map them back to kebab-case when comparing against schema-defined names.

20.1 Schema Validity Constraints

A schema is itself a TEL document — specifically, a TEL document conforming to the `tel-schema` schema (§20.5). It is therefore subject to the same two layers of error reporting as any other TEL document:

- **Parsing and validation against tel-schema.** The E1xx (parsing) and E3xx (validation) taxonomies of §19.3 apply to the schema document directly. Structural malformations — a `field` line appearing inside a `scalar`’s body, an atom in a position where the schema expects a compound child, a malformed identifier in a keyword slot, and similar — are caught here. The error against `tel-schema` is the only signal a schema author receives for this class of mistake. In particular, an unknown child keyword (such as `field` inside `scalar-body`) raises **E306** and a non-Struct compound being given children raises **E301**.
- **Semantic validity of the resulting Schema value.** Once `tel-schema` has accepted the document and `construct_schema` (§20.6) has produced a Schema value, the constraints below apply. They catch errors that the type assignment against `tel-schema` cannot see — properties of the assembled Schema model itself, such as duplicate Definition names, malformed layer merges, or references to undefined types.

A schema is invalid if any of the following holds:

- within a single Struct, the same keyword appears more than once across all members (considering `Field.keyword` and every `Variant.keyword` of the `SelectDefinitions` referenced by `SelectRefs`) (**E201**)
- a `SelectDefinition` has an empty `variants` list (**E202**)
- a `Scalar` has a non-null default and appears in a `Member` whose effective `required` is false (i.e. `Member.required == "loose"`) (**E204**). Absence of a non-required member always means the member is absent; defaults are only meaningful for required members that may be elided in the source document.
- `Schema.sigil` is non-null and is a space, LF, CR, a letter, a control character, a digit, or a parenthetical symbol (§5) (**E208**)
- the keyword `tel` appears as a `Field.keyword` or `Variant.keyword` in any `Struct` or `SelectDefinition` (**E209**)
- a `Reference` names a `TypeName` that does not appear among the `Definitions` of the composed schema (**E210**); a `SelectRef.reference` not appearing as a `SelectDefinition.name` is also E210
- two or more `Definitions` in the *base* `Schema.records` $\cup$ `Schema.scalars` $\cup$ `Schema.selects` share the same name (**E211**). Records, scalars, and selects share a single namespace; cross-kind name collisions in the base are also E211. Same-name `Definitions` across layers trigger `Definition merge` per §20.3 rather than E211.
- an `Exclude(K)` operation in a layer `SelectDefinition` body names a keyword `K` that does not identify any variant of the base `SelectDefinition` (**E212**)
- an `Exclude(K)` operation would empty a `SelectDefinition` referenced by any `SelectRef` whose effective `required` is true (**E213**)
- a layer's `SelectDefinition` contains a variant declaration whose keyword is absent from the base `SelectDefinition` (variant addition is forbidden — would widen the sum) (**E214**)
- a `Reference` resolves to a `SelectDefinition` (a sum at a position expecting a single typed value), or a `SelectRef.reference` resolves to a `RecordDefinition` or `ScalarDefinition` (a non-sum at a position expecting a sum) (**E218**)

Schema Errors (E2xx)

Code	Description	Span
E201	Duplicate keyword within a Struct (across <code>Field</code> keywords and the <code>Variant.keywords</code> of <code>SelectRef</code> -referenced <code>SelectDefinitions</code>)	The second occurrence of the duplicate keyword
E202	<code>SelectDefinition</code> has an empty <code>variants</code> list	The <code>SelectDefinition</code> 's name compound
E203	Root struct has a required atom-assignable member (unreachable: the document root has no atoms)	The required member definition
E204	<code>Scalar</code> has a non-null default but appears in a non-required member	The default field of the <code>Scalar</code>
E205	Two or more Layers within a <code>Schema</code> share the same name	The second Layer with the duplicate name

Code	Description	Span
E206	A Layer's introduced <code>SelectDefinition</code> has a name colliding with an existing <code>Definition</code> in the composed namespace	The overlapping name in the layer
E207	A Layer <code>Field</code> member matches an existing keyword but the base or layer member is not a <code>Field</code> with <code>Struct</code> type	The layer member definition
E208	<code>Schema.sigil</code> is non-null and is a space, LF, CR, a letter, a control character, a digit, or a parenthetical symbol	The <code>sigil</code> field value
E209	The keyword <code>tel</code> appears as a <code>Field.keyword</code> or <code>Variant.keyword</code> in any <code>Struct</code> or <code>SelectDefinition</code> (also §8)	The keyword definition containing <code>tel</code>
E210	A <code>Reference</code> or <code>SelectRef</code> names a <code>TypeName</code> that does not resolve to a <code>Definition</code> in the composed schema	The <code>TypeName</code> atom
E211	Two or more <code>Definitions</code> in the <i>base</i> <code>Schema.records</code> <code>u</code> <code>Schema.scalars</code> <code>u</code> <code>Schema.selects</code> share the same name (any cross-kind name collision is also E211; same-name <code>Definitions</code> across layers merge instead)	The second <code>Definition</code> with the duplicate name
E212	<code>Exclude(K)</code> in a layer's <code>SelectDefinition</code> names a variant <code>K</code> not present in the base <code>SelectDefinition</code>	The <code>Exclude</code> operation's variant keyword
E213	<code>Exclude(K)</code> would empty a <code>SelectDefinition</code> referenced by a required <code>SelectRef</code>	The <code>Exclude</code> operation's variant keyword
E214	A layer's <code>SelectDefinition</code> introduces a variant whose keyword is absent from the base <code>SelectDefinition</code> (variant addition widens the sum)	The offending variant declaration
E215	A layer declares <code>optional</code> against an axis whose merged-base polarity is "default" or "tight" (loosening attempt on required)	The offending field/select declaration
E216	A layer declares <code>repeatable</code> against an axis whose merged-base polarity is "default" or "tight" (loosening attempt on repeatable)	The offending field/select declaration
E217	<code>Exclude</code> appears outside a layer's <code>SelectDefinition</code> body — i.e. inside <code>Schema.document</code> , a <code>RecordDefinition</code> , or a base-side <code>SelectDefinition</code>	The offending <code>exclude</code> compound
E218	A <code>Reference</code> resolves to a <code>SelectDefinition</code> , or a <code>SelectRef</code> . reference resolves to a <code>RecordDefinition</code> / <code>ScalarDefinition</code> (kind mismatch between member shape and resolved <code>Definition</code>)	The offending <code>TypeName</code> atom

20.2 Type Assignment Algorithm

Type assignment translates the presentation model into the semantic model by ascribing a type to every atom and compound node in the tree. It proceeds as a recursive descent over the tree, guided by the schema.

Reference resolution. Wherever this algorithm asks “is T a Struct?”, “is T a Scalar?”, etc., the question is asked of the type T after **reference resolution**: if T is a `Reference(N)`, T is replaced by the `Struct` formed from the members of the matching `RecordDefinition`, or by the `Scalar` formed from the validators of the matching `ScalarDefinition`. A `Reference(N)` resolving to a `SelectDefinition` is **E218** (a sum cannot inhabit a single-typed position).

`SelectRef` resolution is parallel: a `SelectRef(N)` resolves to the variants of the matching `SelectDefinition`; a `SelectRef` resolving to a `RecordDefinition` or `ScalarDefinition` is **E218**. The polarity of the `SelectRef` remains attached to the use site; only the variants are drawn from the named `SelectDefinition`.

Reference and `SelectRef` resolution are single-step (Definitions never name another `Reference/SelectRef` as their direct content); no chasing of resolution chains is required.

Recursive types and depth limit. A Definition may contain References or `SelectRefs` to itself or to other Definitions that ultimately refer back to it (e.g., a `Tree RecordDefinition` whose children `Field` is `Reference(Tree)`). Such cycles are well-formed: the schema describes an arbitrarily deep structure, and any particular document instantiates it to finite depth. Type assignment processes nodes lazily — each compound node descends one level before resolving the next `Reference` — so circularity is naturally bounded by the document’s actual nesting depth.

Nevertheless, a conforming implementation **MUST** defend against pathological inputs by enforcing a recursion-depth limit during type assignment. The **RECOMMENDED** limit is **256** levels of compound nesting. Exceeding this limit is a parser-internal resource error: the implementation reports it with a clear diagnostic (e.g., “document nesting exceeds the configured limit of 256”) and stops type assignment. This is not assigned a TEL error code (E1xx/E2xx/E3xx) because it is not a property of the document itself — a deeper limit would accept the same input. Implementations **MAY** expose the limit as a configurable parameter.

Atom-assignable members. The atom-assignable predicate used throughout this section is the one defined in §20 (after Reference resolution). A member that is not atom-assignable may only be satisfied by compound children (written with an explicit keyword), not by inline atoms.

Document root. The document root is a virtual compound node with type `Schema.document`. It has no atoms; any required atom-assignable members of the root struct cannot be satisfied (**E203**).

Type assignment for a compound node N with type T:

1. T **MUST** be a `Struct`; if it is not, the document is invalid (**E301**).

2. Construct the keyword map K by iterating T in keyword order: for each entry (keyword, type) at member index i , map keyword $\rightarrow (i, \text{type})$. (Schema validity ensures no duplicate keywords within the same struct.)
3. **Atom phase.** Let $\text{pos} = 0$. For each atom A in $N.\text{atoms}$, in order:
 1. Advance pos while $\text{pos} < \text{len}(T.\text{members})$ AND the member M at pos has effective $\text{required} = \text{false}$ (i.e. M is non-required), AND at least one of the following holds:
 - M is **not atom-assignable** (e.g. a `Field` whose type resolves to a `Struct`, or a `SelectRef` whose referenced `SelectDefinition` has any non-Flag variant). Such a member cannot consume any atom and so is always skipped.
 - M is atom-assignable and is **Flag-shaped** (a `Field` of resolved type `Flag`, or a `SelectRef` whose referenced `SelectDefinition`'s variants all resolve to `Flag`), AND the text of A does not match any of M 's keywords (the `Field`'s keyword for a `Field`, any variant's keyword for a `SelectRef`'s referenced `SelectDefinition`). Skipping is permitted because A clearly is not meant to fill M ; A may match a later member.

A member with a `Scalar`-typed field is **never** skipped: a `Scalar` accepts any text, so there is no “doesn't match” condition that would justify skipping. Each advanced-past member is recorded as absent (subject to the required-member check in step 5).

Worked example. Suppose

$T.\text{members} = [\text{Field}(\text{Flag } a), \text{Field}(\text{Flag } b, \text{required}=\text{false}), \text{Field}(\text{Scalar } c)]$ and $N.\text{atoms} = ["a", "xyz"]$:

- Atom a at $\text{pos}=0$: M is `Field(Flag a)`. Effective required is the default (“default” $\rightarrow \text{true}$). No skipping. A matches a ; assign; increment pos to 1.
 - Atom xyz at $\text{pos}=1$: M is `Field(Flag b, required=false)`. Atom-assignable Flag-shaped, atom text doesn't match b . Skip M ; $\text{pos} = 2$. Now M is `Field(Scalar c)`; `Scalar` consumes any text. Assign xyz to c .
1. If $\text{pos} \geq \text{len}(T.\text{members})$, the document is invalid (**E302**: more atoms than assignable member positions).
 2. Let $M = T.\text{members}[\text{pos}]$. M MUST be atom-assignable; if it is not, the document is invalid (**E303**: atom in non-atom-assignable member position).
 3. Assign A to M :
 - If M is a `SelectRef`, the matched variant is the one (from the referenced `SelectDefinition`) whose keyword equals A 's text; if no variant's keyword matches, the document is invalid (**E304**).
 - If M is a `Field` with `Flag` type, A 's text MUST equal $M.\text{keyword}$; if it does not, the document is invalid (**E305**).
 - If M is a `Field` with `Scalar` type, the type of A is $M.\text{type}$ regardless of A 's text (validation against the named helper method is a separate step, described in §21).
 1. If M is not repeatable, increment pos . If M is repeatable, leave pos unchanged; all subsequent atoms are also assigned to M .

4. **Compound child phase.** Let `current_member = -1` and `seen_members = {}` (empty set). For each compound child `C` in `N.children` (iterating across all blocks in order):
 1. Look up `C.keyword` in `K`. If not found, the document is invalid (**E306**: unrecognized keyword for this parent type).
 2. Let `(i, childType) = K[C.keyword]`. The type of `C` is `childType`.
 3. If `i ≠ current_member`: if `i` is in `seen_members`, the document is invalid (**E309**: member children not contiguous); otherwise add `current_member` to `seen_members` (if ≥ 0) and set `current_member = i`.
 4. Record that member at index `i` has been filled by `C`.
 5. Recursively apply type assignment to `C` with type `childType`.
5. **Constraint check.** For each member `M` in `T.members`:
 1. Let `fill_count = (number of atoms assigned to M) + (number of compound children assigned to M)`.
 2. If `M.required` and `fill_count = 0`: if `M` is a `Field` whose type is a `Scalar` with a non-null default, the default value is used as the semantic value and no error is raised; otherwise, the document is invalid (**E307**: required member absent and no default available).
 3. If not `M.repeatable` and `fill_count > 1`, the document is invalid (**E308**: non-repeatable member filled more than once).

20.3 Schema Layering

A Schema MAY include one or more `Layer` values in its `layers` list. Each layer describes an incremental refinement of the schema. The refinements that a layer may perform correspond exactly to the operations that produce a **subtype** of the base — additional information on the record (product) side, and fewer alternatives on the select (sum) side. A composed schema is a subtype of its base schema in the Liskov sense: every document valid under the composed schema, projected back to the base schema's keywords, is valid under the base. §24 gives the formal subtype relation `<`; and proves that every layer-permitted operation is subtype-producing; this subsection states the permitted operations and the merge procedure in prose.

Permitted Operations

A layer MAY apply the following operations to the base, each of which preserves subtyping. The list is organised by the structural duality between records and selects.

Record-side (product) operations:

- **Add a Field** to a `Struct` (root `Struct` or `RecordDefinition` body). The new keyword MUST NOT collide with any keyword already present in the merged `Struct` (**E206**).

- **Add a SelectRef** to a Struct. The referenced SelectDefinition's variant keywords MUST NOT collide with any keyword already present in the merged Struct (**E206**).
- **Refine a Struct in place** (Field merge). When a layer declares a Field whose keyword matches an existing Field and the types are structurally equal (or both Struct, which are merged recursively), the layer's declaration MAY tighten the polarity on either axis by declaring required (tightens the required axis from "loose" to "tight") and/or irrepeatable (tightens the repeatable axis from "loose" to "tight"). The merge also re-runs at any nested Struct level (additional Fields on the layer side are appended). A type mismatch is **E207**.
- **Refine a SelectRef in place**. When a layer declares a SelectRef whose reference matches an existing SelectRef in the same Struct, polarity is merged per MergePolarity; the reference itself does not change.
- **Refine a RecordDefinition in place** (RecordDefinition merge). When a layer declares a record whose name matches an existing RecordDefinition, the layer's members are merged into the existing Definition's members using the same algorithm as Struct merge.

Select-side (sum) operations:

- **Exclude a variant** from a SelectDefinition. Inside a layer's select N body, an Exclude(K) operation removes the variant with keyword K from the merged SelectDefinition. K MUST identify a variant of the base SelectDefinition (**E212**); the exclusion MUST leave at least one variant present if any composed-schema SelectRef whose effective required is true references the SelectDefinition (**E213**).
- **Refine a SelectDefinition in place** (SelectDefinition merge). When a layer declares a select whose name matches an existing SelectDefinition, the layer's Exclude(K) operations and validate lines apply against the base. Variant additions in this position are forbidden (**E214** — would widen the sum).

Common-to-both operations:

- **Add a Definition** (record, scalar, or select) to the composed namespace, when the name does not already exist in the merged namespace.
- **Refine a ScalarDefinition in place**. A same-name scalar in a layer appends validators to the base's validators list (in source order, deduplicated).
- **Append validators** to a Struct, RecordDefinition, or SelectDefinition. A layer's validate K lines at any such position are appended (in source order, deduplicated) to the base's validators list; validators apply in declaration order and AND-conjoined (§21.1), so any layer-added validator runs *after* the base's validators.

The duality is: record-side adds members (extension; subtype-producing for products); select-side excludes variants (narrowing; subtype-producing for sums). Both refine the definition; the directions are mirror-image as type theory requires.

Forbidden Operations

The following operations break subtyping and are NOT permitted in a layer; an implementation detecting any of them MUST report the corresponding error:

- Remove a Field from a Struct (no syntax — structurally prevented).
- Add a variant to an existing SelectDefinition (**E214** — would widen the sum). Introducing a *fresh* SelectDefinition with a fresh name is permitted; what is

forbidden is a variant `K` appearing inside a layer's `select N` body when `K` is not already a base variant of `N`.

- **Loosen required** — declaring optional for an axis whose merged polarity is "default" or "tight" (**E215**).
- **Loosen repeatable** — declaring repeatable for an axis whose merged polarity is "default" or "tight" (**E216**).
- Remove a validator from a `Struct`, `Scalar`, `RecordDefinition`, or `SelectDefinition` (validators are append-only across layers; a layer's `validate K` adds `K`, never removes an existing `K`).
- Change a `Scalar`'s `default`, or a `Field`'s declared `Type` to a structurally different type that is not a Struct-to-Struct refinement.

Composed Schema Identity

A composed schema is identified by the base schema's name together with the ordered sequence of layer names applied to it. Two schemas with the same base name but different layer sequences are distinct schemas. Layer order is part of identity even when reordered layers produce a semantically equivalent composed `Struct`: the BinTEL signature (BinTEL §8) encodes the layer order, so two compositions of the same set of layers in different orders have distinct signatures.

Composed Definition Namespace

The composed schema's Definition namespace is built by walking, in order, the base schema's `Schema.records`, `Schema.scalars`, and `Schema.selects`, then for each layer in layer order its `Layer.records`, `Layer.scalars`, and `Layer.selects`. Definition merge (above) applies whenever a later declaration shares a name with an earlier one. Records, scalars, and selects share the namespace: a layer-introduced Definition whose name collides with an existing Definition of a *different kind* is **E211** in the composed schema. After composition, References and SelectRefs in the base, in any layer, or in any merged member may resolve to any Definition (of the appropriate kind) in the composed namespace.

Layer Body

A `Layer` value has `name`, `overlay`, `records`, `scalars`, and `selects` fields (see §20). In TEL source, a layer's `overlay` is OPTIONAL — a layer that introduces or refines only Definitions without modifying the document root MAY omit `overlay` entirely. When `overlay` is absent it is treated as an empty `Struct` (no members).

Merge Algorithm

The function `MergeStruct(base: Struct, layer: Struct): Struct` produces a new `Struct` that incorporates the layer's members into the base:

1. Begin with a copy of `base.members` in member order.
2. Construct the keyword map `K` for the base struct by iterating it in keyword order: for each entry (`keyword`, `type`) at member index `i`, map `keyword` → (`i`, `members[i]`).

3. For each member L declared by the layer at this Struct position, in source order:

1. **Field members.** If L is a Field with keyword W:

- Look up W in K.
- **Found:** Let (i, M) = K[W]. M MUST be a Field and both M.type and L.type (after Reference resolution) MUST be Struct; if either is not, the layer is invalid (E207). The merged Field at index i has:
 - keyword = W (unchanged);
 - type = MergeStruct(M.type, L.type);
 - per-axis polarities computed by MergePolarity(M.required, L.required) and MergePolarity(M.repeatability, L.repeatability);
 - default = base's default (a layer may not change the default; see Forbidden Operations).
- **Not found:** Append L as a new member at the end of the member list. Add W → (new index, L) to K. (Polarity merge does not apply: a freshly-introduced Field carries its declared polarity directly.)

2. **SelectRef members.** If L is a SelectRef referencing N:

- Resolve N in the composed Definition namespace to a SelectDefinition (E210/E218 if it cannot be resolved to a SelectDefinition).
- For each variant keyword W of the referenced SelectDefinition, look up W in K. If any W matches an existing entry that is *not* a SelectRef referencing the same N, the layer is invalid (E206). If every matched W resolves to a SelectRef referencing the same N, the layer's SelectRef refines the existing one: per-axis polarities are merged via MergePolarity, and the merged SelectRef replaces the base member at that position.
- Otherwise (no overlap at all), append L as a new member at the end of the member list. For each variant V of N, add V.keyword → (new index, L) to K.

3. **Exclude in a Struct position.** An Exclude(K) member MUST NOT appear inside a Struct (root or RecordDefinition body) — only inside a SelectDefinition body (E217).

4. Return the resulting member list as the merged Struct's members. The merged Struct's validators is the concatenation of the base's validators with any new validator names contributed by the layer at this Struct position (in source order, deduplicated). Validators are append-only across layers; a layer MUST NOT remove a base validator.

MergePolarity(base: Polarity, layer: Polarity): Polarity is defined per axis:

base layer	"default"	"loose"	"tight"
"default"	"default"	E215 / E216	"tight"
"loose"	"loose"	"loose" (redundant)	"tight"
"tight"	"tight"	E215 / E216	"tight"

The error code is E215 when the axis is required, E216 when it is repeatable. The rule captures the subtyping direction: tightening (or restating) the cardinality is always allowed; loosening an already-tight or default cardinality is rejected.

MergeRecord(base, layer): RecordDefinition applies MergeStruct to the Definitions' member lists and merges validators by the append-and-deduplicate rule.

MergeScalar(base, layer): ScalarDefinition merges the validators lists by the append-and-deduplicate rule.

MergeSelect(base, layer): SelectDefinition merges a layer's SelectDefinition body into the base SelectDefinition:

1. Begin with a copy of base.variants in declaration order.
2. For each member L in the layer's SelectDefinition body, in source order:
 - If L is a variant declaration with keyword W: W MUST identify an existing variant in the current variant list; if not, the layer is invalid (**E214** — variant addition is forbidden). If W is present and the layer's Variant.type is structurally equal to the base variant's type, the operation is a no-op restatement (permitted); a type mismatch is **E207**.
 - If L is Exclude(W): W MUST identify an existing variant in the current variant list; if not, the layer is invalid (**E212**). Remove the variant from the list.
3. After all layer operations apply, the variant list MUST be non-empty *if* any composed-schema SelectRef whose effective required is true references this SelectDefinition; otherwise **E213**. (A SelectDefinition referenced only by non-required SelectRefs MAY be emptied; the referencing SelectRefs are then effectively unreachable in composed documents.)
4. The merged SelectDefinition's validators is the concatenation of the base's validators with any new validator names contributed by the layer's validate lines (in source order, deduplicated).

A layer attempting to merge a Definition of one kind onto a Definition of another kind (e.g. a layer's select N onto a base record N) is **E211**.

Composing Layers

To apply a sequence of layers $[L_1, L_2, \dots, L_n]$ to a base schema with root Struct R, record list T (for records), scalar list S, and select list U:

1. Let $R_0 = R, T_0 = T, S_0 = S, U_0 = U$.
2. For each L_i in turn:
 - For each record def in L_i .records, in source order:
 - If def.name matches a name in T_{i-1} , replace that RecordDefinition with MergeRecord(existing, def). If it matches a name in S_{i-1} or U_{i-1} , the schema is invalid (**E211**). Otherwise append def to T_i .
 - For each scalar def in L_i .scalars, in source order:
 - If def.name matches a name in S_{i-1} , replace that ScalarDefinition with MergeScalar(existing, def). If it matches a name in T_{i-1} or U_{i-1} , the schema is invalid (**E211**). Otherwise append def to S_i .
 - For each select def in L_i .selects, in source order:
 - If def.name matches a name in U_{i-1} , replace that SelectDefinition with MergeSelect(existing, def). If it matches a name in T_{i-1} or S_{i-1} , the schema is invalid (**E211**). Otherwise append def to U_i .
 - Set $R_i = \text{MergeStruct}(R_{i-1}, L_i.\text{overlay})$.
3. The final R_n is the root Struct of the composed schema; T_n, S_n, U_n are its Definition lists.

Layer Validity Constraints

A schema is invalid if any of the following holds:

- Two or more layers within the same schema share the same name (**E205**).
- A layer's operation triggers any of **E206**, **E207**, **E211**, **E212**, **E213**, **E214**, **E215**, **E216**, **E217**, or **E218** at composition time.
- Within the base schema's own `Schema.records` $\cup$ `Schema.scalars` $\cup$ `Schema.selects`, two or more Definitions share the same name (**E211**). (Within layers, same-name Definitions trigger Definition merge per the rules above, not E211.)

20.4 BinTEL

The binary encoding of the semantic model, BinTEL, is defined in the companion [BinTEL Specification](#). BinTEL provides deterministic serialization of typed TEL documents and defines the schema signature and value hash constructions used for schema identification (§8.1) and compatibility checking (§8.2).

When a BinTEL document is to be carried in a text-oriented context, it MAY be encoded as Unicode text using [BASE-256](#), defined as a companion specification. The BASE-256 textual form is character-for-byte with the BinTEL byte sequence and is recovered losslessly by the BASE-256 decoder.

20.5 The tel-schema Schema

The concrete TEL representation of the schema model defined in §20 is itself a schema, identified by `Schema.name = tel-schema`. The full document is supplied as the file [tel-schema.tel](#) at the root of this repository; this subsection specifies the keyword vocabulary used by that document and states the self-describing closure property.

Vocabulary. The following keywords are used in a schema TEL document. The keywords are themselves kebab-case identifiers (they appear in user-written TEL source); the name of each top-level Definition (a record, scalar, or select) is a **PascalCase TypeName** (§20.7), not a kebab-case identifier. The name of a record, scalar, select, or layer, and the name of the top-level Schema, are typically written **implicitly** — i.e. as the first inline atom of the parent compound (record `Field`, scalar `Email`, select `Status`, layer `auth`, ...) rather than as an explicit name `<value> child` — by the atom/compound interchangeability rule of §19.1. The explicit name `<value> child` compound form is also accepted; both forms produce the same name value in the Schema/Layer/Definition. `tel-schema.tel` uses the implicit form throughout.

TEL keyword	§20 construct
name	<code>Schema.name</code> , <code>Layer.name</code> , or a Definition's name (parent context determines which).
document	<code>Schema.document</code>
layer	<code>Schema.layers[i]</code>
sigil	<code>Schema.sigil</code>
record	

TEL keyword	§20 construct
	A RecordDefinition: <code>Schema.records[i]</code> at schema root, or <code>Layer.records[i]</code> inside a layer compound. The first inline atom is the record's TypeName.
scalar	A ScalarDefinition: <code>Schema.scalars[i]</code> at schema root, or <code>Layer.scalars[i]</code> inside a layer. First inline atom is the TypeName; the body is one or more <code>validate <name></code> lines.
select	At top level (or inside layer): a SelectDefinition — <code>Schema.selects[i]</code> or <code>Layer.selects[i]</code> . First inline atom is the TypeName; body is variant, validate, and (in layers only) <code>exclude</code> lines. At a member position (inside a record body, the document block, or a layer's overlay block): a SelectRef — the first inline atom is the TypeName of the referenced SelectDefinition; polarity is per use site. There is no inline-anonymous form; every select is named.
overlay	<code>Layer.overlay</code> — the struct whose members are merged into the composed document root by the algorithm in §20.3.
field	A Field member. Lives inside a record body, the document block, or a layer's overlay block.
optional	Loosens to required: "loose" (Flag, base-side).
required	Tightens to required: "tight" (Flag, layer-side override of optional). Permitted but redundant in a base, since the default is already tight.
repeatable	Loosens to repeatable: "loose" (Flag, base-side).
irrepeatable	Tightens to repeatable: "tight" (Flag, layer-side override of repeatable). Permitted but redundant in a base, since the default is already tight.
keyword	<code>Field.keyword</code> , <code>Variant.keyword</code> . Carried as the first inline atom of a field or variant compound (or, less commonly, as an explicit keyword <code><text></code> child compound). Kebab-case.
variant	A Variant of a SelectDefinition. First inline atom is the variant's kebab-case keyword; second inline atom is the TypeName of its type.
type	The type-name field of a Field, a Variant, or a SelectRef. The value is a TypeName resolving (via §20.2 reference resolution) to either a user-declared Definition or a built-in type (Flag, String, Identifier, Sigil).
validate	Inside a scalar body, names a scalar validator. Inside a record body, a select body, the document block, or an overlay, names a struct-level (or select-level) validator (§21.6). The shared-namespace rule of §21.1 means the same name MAY be used in different contexts.
default	<code>Field.default</code> — the value used when a required Scalar-typed field is absent from the document. Valid only on required Scalar-typed fields (E204 otherwise).
exclude	A layer-only operation (§20.3) that excludes a variant from the merged SelectDefinition. Its inline atom is the kebab-case

TEL keyword	§20 construct
	keyword K of the variant to exclude. Permitted only inside a select body within a layer compound (E217 otherwise).

Predefined type names. The names `Flag`, `String`, `Identifier`, and `Sigil` are predefined (in `TypeName` form, i.e. `PascalCase`) and resolve to the built-in `Flag` type and the three built-in scalar validators (§21.5). User schemas MAY NOT declare a Definition with any of these names (collision is **E211**).

Reserved keywords. Only `tel` is universally reserved across all TEL documents (**E209**, §8). The other keywords listed above are part of the tel-schema vocabulary and have meaning only when a TEL document is being parsed as a *schema document* — i.e. when its schema is the tel-schema. They do not constrain user-defined schemas: a user schema may freely define `Field.keyword` or `Variant.keyword` values such as `name`, `document`, `layer`, `record`, `scalar`, `select`, etc., because the validity check applied to a user document is against the user schema’s keyword set, not against the schema-language vocabulary.

Member ordering and inline syntax. Member order in `Field` is keyword, type (both required Scalars; type carries a `TypeName`), then the four loosen/tighten flags (`optional`, `required`, `repeatable`, `irrepeatable`), then `default` (optional Scalar). The atom-phase rules of §20.2 / §20.8 let a typical field declaration fit a single line: the first atom is the keyword, the second atom is the type-name, each subsequent flag-matching atom toggles its flag, and any remaining non-flag atom fills `default`. For example,

```
field country String optional unknown
```

declares an optional `country` field of type `String` (the built-in scalar) with default value `unknown`. The convention is **flags before default**: `default` is the last optional Scalar and so consumes the first non-flag atom after the type-name. Variants follow the same pattern but carry only keyword and type, e.g. `variant active Flag`. There are no marker keywords: position determines meaning.

A `SelectRef` at a member position has the inline shape `select <TypeName> [polarity]`:

```
select Status optional
```

introduces a non-required `SelectRef` referencing the `Status` `SelectDefinition`.

References. A `Field.type`, `Variant.type`, or `SelectRef.reference` resolves through the composed namespace described above. If the name is `Flag`, `String`, `Identifier`, or `Sigil` it short-circuits to the built-in. Otherwise it must match a `record`, `scalar`, or `select` declared somewhere in the composed schema (base or any layer); failing that, the schema is invalid with **E210**, or with **E218** if the kind doesn’t match the position (a `Field.type` resolving to a `SelectDefinition`, or a `SelectRef.reference` resolving to a `RecordDefinition` or `ScalarDefinition`). References may form cycles via `record` or `select` definitions in their resolved bodies — the natural case for recursive data.

Self-describing closure and bootstrap. `tel-schema.tel` MUST be a valid TEL document when parsed under the schema it itself defines. To break the regress that would otherwise prevent a schema document from being parsed at all, **every**

conforming TEL parser MUST embed the tel-schema schema as a built-in, available before any external schema has been resolved. When a TEL document’s pragma identifies tel-schema as its schema, the parser uses the built-in form rather than performing schema retrieval. The built-in MUST produce the same Schema model that would result from parsing the canonical tel-schema.tel under itself, and MUST produce a byte-identical BinTEL encoding of that schema and the same SHA-256 value hash (§3 of the BinTEL Specification). The hash is normative — two conforming implementations MUST agree on it.

The pinned value, computed against the canonical tel-schema.tel in this repository, is:

Form	Value
SHA-256	9033cf054ed14fc460cfd04502a2b69e1ac840cd1035f213492b74af7df2a8dd
BASE-256	Γ3KqNÖ0τŠKÐEḤḤÇΞΤψℓúŘ5ỲΓIbtíŷỲzḄ

The BinTEL document root encoding of tel-schema.tel is 887 bytes; the raw bytes are recorded in demo/tel-schema.bintel.hex and the hash in demo/tel-schema.hash. The same value is pinned in §3 of the BinTEL Specification.

Verifying the built-in. An implementation’s built-in tel-schema Schema value (the “axiom”) is a hand-written construction; it is easy to introduce silent drift between the axiom and the canonical tel-schema.tel. Conforming implementations SHOULD therefore include two self-consistency checks:

- **Structural-equality check.** Parse tel-schema.tel against the axiom and run construct_schema (§20.6) on the result; assert that the constructed Schema is structurally equal to the axiom (modulo the built-in scalars Identifier, TypeName, Sigil, String, which an implementation may inject into the axiom but which construct_schema will not produce from the document).
- **Value-hash check.** Encode the axiom (or, equivalently, the parsed-and-constructed document) as BinTEL and compute its SHA-256; assert that the result equals the pinned value above. This is the content-addressed counterpart to the structural check.

Pinning both invariants in tandem makes axiom drift very hard to introduce undetected.

20.6 Schema Construction from the Semantic Model

Type assignment (§20.2) produces a tree of Element values typed by the tel-schema schema. To obtain a Schema interface instance, an implementation traverses this tree and populates the fields of the Schema model:

1. **Schema root.** Create a Schema value. Iterate the root element’s children in **source order** (the order in which they appear in the document):
 - For the name child (Scalar with validator identifier), set Schema.name to its text.
 - For the sigil child, set Schema.sigil to its text (or leave null if absent).
 - For each record child, append a RecordDefinition to Schema.records constructed per step 2. The resulting list preserves source order.

- For each scalar child, append a `ScalarDefinition` to `Schema.scalars` constructed per step 2b. The resulting list preserves source order.
- For each select child (at schema root, i.e. **outside** any record/document/overlay body), append a `SelectDefinition` to `Schema.selects` constructed per step 2c. The resulting list preserves source order.
- For the document child, set `Schema.document` to the `Struct` built from the document element's children per step 6.
- For each layer child, append a `Layer` to `Schema.layers` constructed per step 4. The resulting list preserves source order — layer composition (§20.3) applies layers in this order.

2. **RecordDefinition construction.** From a record element: take the name child (or first inline atom, which **MUST** be a `TypeName`) as `RecordDefinition.name`; build `RecordDefinition.members` and `RecordDefinition.validators` from the element's remaining children per step 6. 2b. **ScalarDefinition construction.** From a scalar element: take the name child (or first inline atom, a `TypeName`) as `ScalarDefinition.name`; for each validate child within the element, append the child's inline-atom text to `ScalarDefinition.validators`, in source order. 2c. **SelectDefinition construction.** From a top-level select element: take the name child (or first inline atom, a `TypeName`) as `SelectDefinition.name`; for each variant child, append a `Variant` to `SelectDefinition.variants` per step 3; for each validate child, append the child's inline-atom text to `SelectDefinition.validators`; for each exclude child (permitted only inside a layer's select body — otherwise **E217**), append a `Member::Exclude(K)` entry to the `SelectDefinition` body that will be consumed by `MergeSelect` during layer composition.

3. **Member construction.** A field element becomes a `Field`; a select element at a member position (inside a record body, document, or overlay) becomes a `SelectRef`; a variant element (inside a top-level select) becomes a `Variant`. The four loosen/tighten Flag children (`optional`, `required`, `repeatable`, `irrepeatable`), where applicable, collectively determine the two Polarity fields:

- `required` =
 - `"tight"` if the `required` Flag is present;
 - `"loose"` if the `optional` Flag is present;
 - `"default"`.
- `repeatable` =
 - `"tight"` if the `irrepeatable` Flag is present;
 - `"loose"` if the `repeatable` Flag is present;
 - `"default"`.

The tightening flag (`required`, `irrepeatable`) wins when both flags on an axis are present — redundant declarations are benign no-ops.

Within a `Field`:

- keyword child → `Field.keyword` (kebab-case identifier).
- The four loosen/tighten Flag children compute `Field.required` and `Field.repeatable`.
- The type Scalar child or atom → `Field.type` as a `Reference(TypeName)` (per step 5).

- The optional default Scalar child or atom → `Field.default` (a string), or null if absent.

Within a `SelectRef`:

- The four loosen/tighten Flag children compute `SelectRef.required` and `SelectRef.repeatable`.
- The first inline atom (or the name / type child compound; the schema-of-schemas uses the first inline atom) is a `TypeName` and becomes `SelectRef.reference`.

Within a `Variant`:

- keyword child or first inline atom → `Variant.keyword` (kebab-case).
- type child or second inline atom → `Variant.type` as a `Reference(TypeName)`.

- 4. Layer construction.** From a layer element: take the name child as `Layer.name`; build `Layer.overlay` from the overlay element's children per step 6 (treating an absent overlay as an empty Struct); construct `Layer.records` from each record child within the layer (step 2); `Layer.scalars` from each scalar child (step 2b); `Layer.selects` from each select child at the layer's top level (step 2c, which inside a layer body permits exclude children).
- 5. Type construction.** Every `Field.type` and `Variant.type` is a `Reference` whose name is the inline-atom (or type child-compound) text, a `TypeName`. Resolution of a `Reference` happens at type assignment time (§20.2) and selects either a `RecordDefinition`'s `Struct`, a `ScalarDefinition`'s `Scalar`, or one of the four built-in types (`Flag`, `String`, `Identifier`, `Sigil`). A `Reference` resolving to a `SelectDefinition` is **E218**.
- 6. Struct construction.** Given the children of a Struct-shaped compound (the document element, a layer element's overlay, or a record element's body), produce a `Struct` (or, for record, a `RecordDefinition` whose members and validators are taken from this step):
 - Each field child contributes one `Member::Field` constructed per step 3.
 - Each select child at this position contributes one `Member::SelectRef` constructed per step 3.
 - Each validate child contributes its inline-atom text to the resulting validators list, in source order.
 - An exclude child is **not** permitted in this position (**E217**); exclude lives only inside a layer's select body, where it is consumed by `MergeSelect` (§20.3).

Schema construction **MUST** be deterministic: two implementations applied to the same input **MUST** produce identical Schema values. After construction, the resulting schema is checked against the validity constraints of §20.1 and §20.3; any failure is reported as the corresponding **E2xx** error.

20.7 Identifier Grammars

This specification uses two ASCII identifier kinds, distinguished by initial-letter case.

Kebab-case identifier. Used for non-type names: schema names, layer names, field keywords, variant keywords, and validator names.

```
identifier ::= lower-letter ( '-'? lower-letter-or-digit )*  
lower-letter ::= [a-z]  
lower-letter-or-digit ::= [a-z] | [0-9]
```

That is:

- An identifier **MUST** begin with a lowercase ASCII letter.
- It **MAY** contain lowercase letters, digits, and hyphens.
- Hyphens **MUST NOT** appear consecutively (no --).
- Hyphens **MUST NOT** appear at the start or end of the identifier.
- The empty string is not a valid identifier.

This grammar is enforced by the built-in `Identifier` validator (§21.5).

PascalCase TypeName. Used for the name of every Definition (record, scalar, select) and for every Reference / SelectRef target.

```
type-name      ::= upper-letter ( letter-or-digit )*  
upper-letter   ::= [A-Z]  
letter-or-digit ::= [A-Z] | [a-z] | [0-9]
```

That is:

- A `TypeName` **MUST** begin with an uppercase ASCII letter.
- It **MAY** contain uppercase letters, lowercase letters, and digits.
- It **MUST NOT** contain hyphens, underscores, or non-ASCII characters.
- The empty string is not a valid `TypeName`.

This grammar is enforced by the built-in `TypeName` validator (§21.5). The two grammars are disjoint: every well-formed identifier begins with a lowercase letter, every well-formed `TypeName` begins with an uppercase letter, so a single atom's lexical form tells the reader which kind it is.

20.8 Footnote: Inline-Atom Position Constraints

The atom phase in §20.2 allows skipping past non-required `Flag`-shaped members when an atom does not match their keyword. It does **not** allow skipping past non-required `Scalar` members, because a `Scalar` accepts any atom text — there is no “doesn't match” condition that would justify a skip. As a consequence, when a struct's atom-assignable members include two or more non-required `Scalar Fields`, only their prefix can be filled positionally with inline atoms; later optional `Scalars` in the same struct must be filled via explicit compound children. Schema authors should either order their members so this limitation does not arise, or expect users to write the later optional `Scalars` as child compounds (e.g. `field-name value`).

21. Validation

Type assignment (§20.2) ascribes a `Type` to every node. For `Flag` types, the structure of the document is sufficient to determine validity. For `Scalar` and `Struct` types, the value or structure MAY additionally be checked against one or more named **validators** declared in the schema.

21.1 Validators

A **validator** is a named helper method that examines a value and returns `Valid` or `Invalid`. A validator can be applied to a `Scalar` value (in which case it inspects the atom's text), a `Struct` element (in which case it inspects the struct's children), or a `SelectDefinition` instance (in which case it inspects the chosen variant). `Struct` and `SelectDefinition` requests share the `kind = "struct"` shape (§21.2) — they are the same wire form, distinguished by the parent type at the call site.

- A `Scalar` member's `validators` list (§20) names the validators applied to that scalar's value.
- A `Struct`'s `validators` list (§20) names the validators applied to that struct.
- A `SelectDefinition`'s `validators` list (§20) names the validators applied to instances of that sum (whichever variant is chosen).

When a value or struct has more than one validator, they apply in **AND-conjunction**: every validator **MUST** return `Valid` for the value to be accepted. The validators are invoked in the order they are listed; an implementation **MAY** short-circuit on the first `Invalid` response or invoke every validator regardless (the document is invalid in either case if any returns `Invalid`).

Validators live in a **single shared namespace**: a name like `non-empty` refers to one helper, which is dispatched on the request kind (scalar or struct) at invocation time. A helper that supports only one kind returns `Invalid` with an appropriate message when invoked on the other.

This specification mandates only the three built-in validators required by `tel-schema` itself (§21.5). Every other validator is application-defined; a parser is configured with a callback (§21.4) that resolves each validator name to a concrete check.

As illustrations:

- A scalar validator named `ipv4` accepts dotted-quad IPv4 address literals. A schema field whose `scalar validators` list contains `ipv4` will be invoked once per atom text encountered for that field; an invalid octet produces an `Invalid` response with a `Diagnostic::Scalar` pointing into the offending part of the text.
- A struct validator named `postcode-required-when-uk` examines an address struct's children and rejects the struct if `country == "UK"` but `postcode` is absent. It produces a `Diagnostic::Struct` whose `fields` map points at the missing `postcode`.

21.2 Request and Response

The request and response of a helper method invocation are defined by the following types:

```
type ValidationRequest =  
  | { method: string; kind: "scalar"; value: string }  
  | { method: string; kind: "struct"; element: StructElement };  
  
type ValidationResponse = Valid | Invalid;  
  
interface Valid {}  
  
interface Invalid {  
  diagnostic: Diagnostic;  
}  
  
type Diagnostic =  
  | { kind: "scalar"; message: string; span?: { start: number;  
    end: number } }  
  | { kind: "struct"; message: string; fields?: { [keyword:  
    string]: Diagnostic } };
```

StructElement is the semantic-model node (§18.2) being validated — its keyword index, its schema type (after Reference resolution), and its child elements. The validator MAY traverse its children to inspect values, sub-structs, and flag presence.

Kind matching. A helper method MUST respect the request's kind: a { kind: "scalar" } request returns Valid or Invalid whose diagnostic.kind == "scalar"; a { kind: "struct" } request returns Valid or Invalid whose diagnostic.kind == "struct". Mismatched kinds are a contract violation.

Diagnostic shape.

- `D diagnostic::Scalar` carries a message and an OPTIONAL span. When present, span is a half-open range [start, end) of zero-based code-point indices into the value's input text. start and end MUST both be present or both absent — partial spans are forbidden.
- `D diagnostic::Struct` carries a message and an OPTIONAL fields map keyed by child keyword (Field keyword or Select variant keyword). The nested Diagnostic for each entry MUST match the schema-declared type of that child: a `D diagnostic::Scalar` for a Scalar child, a `D diagnostic::Struct` for a Struct child, or a `D diagnostic::Scalar` with no span for a Flag child. Spans never appear in `D diagnostic::Struct`; structures address content by keyword path, not by offset.

Top-level message. An Invalid response carries exactly one top-level Diagnostic, with a human-readable message describing the failure as a whole. Per-field detail is conveyed via the recursive fields map (for struct diagnostics) or via the optional span (for scalar diagnostics).

21.3 Reporting and Span Resolution

When a helper method returns `Invalid`, the implementation reports **E310** with the returned `Diagnostic` translated to document-level offsets:

- For a `Diagnostic::Scalar` with a span, the implementation **MUST** translate the value-relative offsets to document offsets by adding the document offset of the start of the value's input text. If the value is the body of a compound's inline atom, the offset is the atom's beginning in the document; for source and literal atoms, the offset is the first content byte of the atom's payload after the delimiter handling defined in §14–§15.
- For a `Diagnostic::Scalar` with no span, the implementation reports the error against the span of the enclosing scalar value's text in the document.
- For a `Diagnostic::Struct`, the implementation reports the error against the keyword span of the enclosing struct compound. If the `fields` map is non-empty, the implementation **SHOULD** recursively descend into each entry: a nested `Diagnostic::Scalar` is reported against the keyword'd child compound's value text (with the optional span applied as above); a nested `Diagnostic::Struct` is reported against the child compound's keyword, and recursed into.

An implementation rendering diagnostics to a user **SHOULD** present the top-level message as the primary headline and the recursive structure as collapsible detail; the choice of presentation is not normative.

Malformed validator responses. A helper method **MUST** return a `Diagnostic` whose kind matches the request's kind: a `Scalar` request gets a `Diagnostic::Scalar`; a `Struct` request gets a `Diagnostic::Struct`. A diagnostic **MUST** also satisfy the structural rules of §21.2 — spans are forbidden in `Diagnostic::Struct`; each entry in a `fields` map carries a `Diagnostic` whose kind matches the schema-declared type of the keyed child; spans on `Diagnostic::Scalar` **MUST** be valid half-open code-point ranges into the value's text (i.e., $0 \leq \text{start} \leq \text{end} \leq \text{codepoint_count}(\text{value})$).

When an implementation detects that a returned diagnostic violates any of these rules, it **MUST** treat the response as a **contract violation**:

1. Discard the malformed diagnostic.
2. Substitute a synthetic `Diagnostic::Scalar` { message: "validator <method> returned a malformed diagnostic", span: None } (for `Scalar` requests) or a synthetic `Diagnostic::Struct` { message: "validator <method> returned a malformed diagnostic", fields: {} } (for `Struct` requests).
3. Report **E310** against the value or struct as if the validator had returned the synthetic diagnostic.

This rule lets a parser cope with buggy validator callbacks without aborting validation of the rest of the document. Implementations **MAY** additionally log the contract violation for debugging.

21.4 Helper Method Binding

A TEL parser that wishes to enable validation **MUST** be provided with a **callback function** that conforms to the helper method interface: given a `ValidationRequest`, it returns a `ValidationResponse`. The parser invokes this callback for each scalar value and each struct element whose schema declares one or more validators. How the callback is supplied is determined by the host language or environment (a function parameter, a trait implementation, an interface injection).

This specification does not prescribe a wire protocol, service discovery mechanism, or serialization format for helper method invocation:

- A parser embedded in an application **MAY** implement the callback directly in the host language.
- An IDE, text editor, or LSP server **MAY** delegate helper method calls to an external service (REST, RPC, subprocess), but the mechanism by which the editor discovers and configures such a service is outside the scope of this specification.

If no callback is provided, the parser **MUST** skip validation entirely (no E310 errors are raised). All other parsing and validation proceeds normally.

21.5 Built-in Validators

This specification does not mandate a portable validator library — applications choose which validators they implement. However, four validators are referenced by the `tel-schema` schema itself (via the four built-in `TypeName` `Flag`, `String`, `Identifier`, `Sigil`, and the internal `TypeName` validator used by `Definition-name` fields) and therefore **MUST** be implemented by any TEL parser that wishes to parse schema documents at all. All are **scalar** validators (`kind = "scalar"`); they return `Invalid` with `Diagnostic::Scalar` on failure.

The validator names defined by this specification are kebab-case (per §20.7 / §21.1's shared namespace), even where they share a base lexeme with a capitalized `TypeName`. The correspondence is:

Built-in <code>TypeName</code>	Validator name (identifier kind)	Meaning at a use site
<code>String</code>	<code>string</code>	Unconstrained scalar value
<code>Identifier</code>	<code>identifier</code>	Kebab-case identifier (§20.7)
<code>Sigil</code>	<code>sigil</code>	Single sigil character
<code>Flag</code>	n/a (Flag is a Type, not a Scalar)	Flag-typed member

A fifth, **type-name**, is required to validate `TypeName` atoms (`Definition` names and `Reference` / `SelectRef` targets). It does not have a corresponding built-in `TypeName` in the schema-of-schemas, because `TypeName` is meta-circular: the field that carries a `TypeName` itself uses the built-in `type-name` validator. Schemas **MAY** use `type-name` directly to validate any user-defined scalar that must carry a type name.

identifier. Accepts a string that conforms to the kebab-case identifier grammar of §20.7 — optionally including leading prime (') characters. On failure, returns a `Diagnostic::Scalar` whose message describes the first violation encountered (e.g. “leading hyphen”, “consecutive hyphens”, “empty identifier”, “non-ASCII or uppercase character”) and whose span covers the offending portion of the input `(l0, len)` if the input is malformed end-to-end).

type-name. Accepts a string that conforms to the PascalCase `TypeName` grammar of §20.7. On failure, returns a `Diagnostic::Scalar` whose message describes the first violation (e.g. “leading lowercase letter”, “hyphen in `TypeName`”, “empty `TypeName`”, “non-ASCII character”) and whose span covers the offending portion of the input.

sigil. Accepts a single-character string whose character satisfies the constraints in §8: it MUST NOT be U+0020 SPACE, U+000A LINE FEED, U+000D CARRIAGE RETURN, an ASCII letter, an ASCII control character, an ASCII digit, or a parenthetical symbol (§5). On failure, returns a `Diagnostic::Scalar` covering the offending input.

string. Accepts any input string without further constraint; equivalent to “no validation”. The `string` validator MUST always return `Valid`. It exists so a schema can declare a field whose validator is the unconstrained string type without the application needing to define a custom validator.

Implementations MAY provide additional validators beyond these. The four above are the minimum required for `tel-schema` parsing to function. None of the four supports the `struct` kind; invoked on a struct request, they return `Invalid` with `Diagnostic::Struct { message: "validator not applicable to struct values", fields: {} }`.

21.6 Struct and Select Validators

A `Struct` (whether it is the root, a `RecordDefinition`’s body, or an overlay’s body) MAY carry a `validators` list (§20). A `SelectDefinition` MAY likewise carry a `validators` list, for cross-cutting checks over which variant has been chosen. When the struct or select appears in a document, validation invokes each named validator AFTER all of the element’s children have themselves been validated. This sequencing means a struct/select validator can rely on its children having well-typed values and on any per-child validators having already passed; if any child validator returned `Invalid`, the struct/select validator MAY still be invoked (the implementation chooses — see §21.4).

Struct validators are particularly useful for **cross-field constraints**: rules that span multiple members of a struct, such as “postcode required when country is UK”, “start date must precede end date”, or “at most one of A, B, C is present”. Such constraints cannot be expressed by scalar validators alone, because each scalar validator sees only its own value.

Select validators inspect the chosen variant: rules such as “this variant is only valid when the surrounding context X is true” or “variant K requires capability Y”. A select validator sees only one variant per invocation (the one the document chose); its `Diagnostic::Struct`’s `fields` map (if present) is keyed by variant keywords, addressing the chosen variant’s content.

A `Diagnostic::Struct` returned by a struct or select validator MUST satisfy the structural rules of §21.2: spans are forbidden at the struct level; per-field detail is conveyed via the recursive `fields` map. The implementation translates the diagnostic per §21.3.

The error code raised for a failing struct or select validator is the same **E310** used for scalar validators (one code, three shapes of diagnostic) — the kind of the diagnostic and the parent type distinguish the cases.

Validation Errors (E3xx)

Code	Description	Span
E301	Compound's type is not a <code>Struct</code>	The compound's keyword
E302	More atoms on a compound than there are assignable member positions	The first excess atom
E303	Atom appears at a member position that is not atom-assignable	The atom
E304	Atom text matches no variant keyword of the <code>SelectDefinition</code> referenced by a <code>SelectRef</code> member	The atom
E305	Atom text does not match a <code>Field</code> member's <code>Flag</code> keyword	The atom
E306	Compound keyword is not recognized for its parent type	The compound's keyword
E307	Required member absent, and member is not a <code>Field</code> with a <code>Scalar</code> type with non-null default	Zero-width span at the end of the parent compound's last child (or at the parent keyword if no children)
E308	Non-repeatable member is filled more than once	The keyword of the second occurrence
E309	Compound children of the same member are not contiguous	The keyword of the non-contiguous child (the second group's first child)
E310	A scalar value or struct element failed validation by a named validator	As resolved by §21.3 from the returned <code>Diagnostic</code>
E311	Flag-typed compound has atoms or compound children	The first atom or child of the <code>Flag</code> compound

22. Reserialization and Editing

The presentation model can be mutated to reflect changes to the semantic model, preserving formatting, comments, tabulations, and remarks wherever possible. Mutations are expressed as operations on the semantic model, which are then reflected in the presentation layer.

There are two categories of editor:

- **Humans**, who modify source text directly with full flexibility and no constraints on what changes may be made
- **Machines** (agents or processors), which apply structured operations to the semantic model and reserialize through the presentation layer

22.1 Comment Attachment and Editing

Each Block in the presentation model carries zero or more attached comments (§11.1) that precede its compounds. These comments travel with the block under programmatic transformations.

When a machine deletes a compound, the Block that contained it is updated. If the deleted compound was the only compound in its block, and if the block has attached comments, those comments are also removed (since their meaning was associated with that block).

When a machine moves a compound, its containing block's structure is preserved where possible: if the move results in the block having no remaining compounds, the block (and any attached comments) moves with the compound to the new location.

When a machine inserts a compound constructed from purely semantic information, no comment is attached to it initially; it is placed into an existing block or a new empty block as appropriate.

22.2 Machine Operations

A machine **MUST** perform only operations drawn from the following set. Each operation preserves all presentation-layer details that are not directly affected by the operation: remarks, trailingBlankLines counts, precedingSpaces on inline atoms, and tabulation marker offsets are all retained unless the operation explicitly targets them.

Operation addressing. Operations identify their target by a **path**. Two path kinds exist:

- A **compound path** is a sequence of (block_index, compound_index) pairs descending from the document root to a specific Compound. Used by update-value, attach-remark, remove-remark, delete, replace, set-flag, unset-flag, insert, insert-before, insert-after, reorder-within-group, and reorder-groups. An empty compound path refers to the (virtual) document root.
- A **block path** is a compound path of length 0 or more (locating the parent compound, or the document root when empty) plus a block_index selecting one of that parent's child blocks. Used by insert-into-block and resize-tabulation, which target an entire block rather than a compound within it.

`construct` is a constructor rather than a tree-mutation operation: it produces a fresh compound from semantic data, with no path argument; the caller subsequently uses `insert`, `insert-before`, `insert-after`, or `insert-into-block` to place the result. `reorder-groups` takes a compound path identifying the parent struct plus two member indices identifying the groups to swap.

Operations that move or remove a compound **MUST** update any in-flight paths that referenced that compound's position; the caller is responsible for invalidating cached paths after a mutation.

Sigil invariant. A machine **MUST NOT** change the document's sigil. The sigil in effect when the document was parsed is preserved exactly in any reserialized output.

Literal atom delimiter invariant. The delimiter of a literal atom **MUST NOT** appear as a line within the atom's payload. When a machine updates the value of a literal atom, it **MUST** check whether the existing delimiter appears verbatim as a line in the new payload. If it does, the editor **MUST** choose a new delimiter that does not appear as a line in the new payload before writing the updated atom.

This specification does not mandate a delimiter-selection algorithm for editor use. Two common strategies are:

- **Dash-extension.** Start from a short fixed delimiter such as `---`; if it appears as a line in the payload, lengthen by one `-` (try `----`, `-----`, ...) until no collision remains. This is the strategy used by canonical serialization (§22.3) and is **RECOMMENDED** for any tool that values diff-stability.
- **High-entropy token.** Generate a random or UUID-derived ASCII identifier and use it as the delimiter (after verifying the no-collision invariant). This single-pass option suits writers that have no need for short, human-readable delimiters.

Any other deterministic strategy that respects the no-collision invariant is also conforming. The choice is an application concern; only canonical serialization (§22.3) is normatively bound to the dash-extension algorithm.

delete — Remove a compound that is not required. Any remark attached to the compound is removed with it. If the compound's block becomes empty (no remaining compounds), the block and its attached comments are also removed.

replace — Substitute a compound for another at the same position in the same block. A replacement is valid if and only if:

- the replacement compound's keyword identifies the same schema member as the original — that is, either both keywords map (via the parent's keyword map `K`) to the same `Field` member, or both map to (possibly different) `Variants` of the same `Select` member; and
- the replacement compound is itself well-typed under the type that `K` maps it to (the new keyword's type after `Reference` resolution); in particular, a `Scalar`-typed replacement **MUST** have its value validate against the target `Scalar`'s helper method (§21).

The replacement retains the original compound's remark and its position within the block. Attached comments on the block are preserved. If the replacement targets a `Select` member and uses a different variant from the original, the keyword in the presentation layer is updated accordingly and the body of the new compound **MUST** match the new variant's type.

construct — Create a new compound from purely semantic information, with no presentation-layer context. The constructed compound carries no remark and has no attached comments. No blank lines appear between its children. No tabulation is added. The canonical presentation form is determined by iterating the struct's members in member order:

1. Starting from the first member, each non-repeatable `Field` member whose type is `Scalar` is serialized as an inline atom, in member order, for as long as consecutive members satisfy this condition and the value can be represented as an inline atom (see the atom-form escalation algorithm in §22.3).
2. If the next member after the initial run of non-repeatable scalars is an all-Flag `Select`, each present flag is serialized as an inline atom.
3. Otherwise, if the next member is a repeatable `Field` whose type is `Scalar`, each occurrence is serialized as an inline atom (if representable per §22.3).
4. All remaining children — including any `Field` members whose type is `Struct`, mixed-variant `Select` members, and any members beyond the first repeatable scalar — are serialized as compound children with explicit keywords.

Atom form escalation. When serializing a `Scalar` value to TEL source, the atom form is selected by the deterministic algorithm defined in §22.3 (the canonical-serialization atom-form rule). **construct** **MUST** apply that algorithm exactly, so that two implementations producing the same compound from the same semantic value emit byte-identical text. The §22.3 algorithm picks the **first** form among inline → source → literal that the value can faithfully carry; the tie-breaking choice is therefore a function of the value's text alone.

If a value cannot be an inline atom, the `Scalar` is serialized as a compound child with an explicit keyword and the appropriate atom body, rather than as an inline atom on the parent line.

Each inline atom uses a single preceding space (`precedingSpaces = 1`). Each compound child is indented by one level (two spaces) relative to its parent.

insert — Insert a compound into the child structure of a parent at the natural position for its member: after all existing compounds of the same member, within the same block if one exists for that member group, or in a new block otherwise.

insert-before — Insert a compound immediately before a specified existing sibling compound. The inserted compound is placed in the same block as the sibling if the block does not have a tabulation, or in a new block immediately before the sibling's block if it does.

insert-after — Insert a compound immediately after a specified existing sibling compound, subject to the same block-placement rules as **insert-before**.

insert-into-block — Append a compound to the compounds list of a specified existing block. This is the natural way to add rows to a tabulated block. The block's tabulation must have sufficient column capacity for the new compound; if not, **resize-tabulation** must be applied first.

attach-remark — Add a remark string to a compound. If the compound already has a remark, it is replaced.

remove-remark — Remove the remark from a compound.

update-value — For a compound or atom whose schema type is `Scalar`, update the atom text to a new string. The new string **MUST** be valid according to the named helper method (§21). All other presentation details of the compound are retained.

set-flag — Add a `Flag`-typed node within a parent, provided the result satisfies the repeatable constraint for that member. The flag is placed as an inline atom if both of the following hold: (a) the flag's member precedes all compound children in member order (i.e., no member that currently has compound children appears earlier), and (b) inserting the atom does not require moving any existing compound children to atom position. If either condition is not met, the flag is placed as a compound child using the **insert** placement rules.

unset-flag — Remove a `Flag`-typed node within a parent, provided the result satisfies the required constraint for that member. If the flag is currently an inline atom, the atom is removed and the `precedingSpaces` of subsequent atoms are preserved. If the flag is currently a compound child, it is removed using the **delete** rules.

reorder-within-group — Change the position of a compound among its siblings within the same member group (i.e., other compounds filling the same schema member). This operation never violates E309. The reordered compound retains its remark; attached comments on the affected blocks are preserved.

reorder-groups — Change the relative order of two distinct member groups within a parent's child structure, by moving all blocks belonging to one group before or after all blocks belonging to another. This is valid as long as neither group is interleaved with the other (E309 is satisfied before and after). Attached comments on all affected blocks are preserved.

resize-tabulation — Adjust the `markerOffsets` of a block's `Tabulation` to accommodate all current column values and any values about to be added. The new offsets are computed by the **minimal-offsets algorithm** below; this is normative — two conforming implementations applied to the same block produce identical `markerOffsets`. After resizing, all existing row content **MUST** be re-padded with spaces to align to the new column positions. The `headings` list is updated in parallel with `markerOffsets`: existing headings are preserved in place and re-padded within their updated column spans; no heading text is added or removed by this operation.

Minimal-offsets algorithm. Given a `Tabulation` with n columns:

1. For each column i in $0..n$, compute w_i = the maximum code-point width of any value that will appear in column i after the operation completes (taking the maximum across every existing row plus every planned new row, including the heading text in column i).

2. The keyword column (column 0) starts at margin offset 0. Each subsequent column's marker starts at a position that leaves the previous column its full w_{i-1} width plus exactly two spaces of inter-column gap, which is the minimum gap required by §16.1 for the hard-space marker separator. Formally: $\text{markerOffsets}[0] = w_0 + 2$, and for $i \geq 1$, $\text{markerOffsets}[i] = \text{markerOffsets}[i-1] + 1 + w_i + 2$ (the +1 accounts for the sigil character at position $\text{markerOffsets}[i-1]$).
3. The result is the unique smallest sequence of offsets that fits every value without violating the hard-space minimum-gap rule of §16.1.

This procedure is deterministic and produces byte-identical offsets across implementations.

22.3 Canonical Document Serialization

A **canonical serialization** of a semantic model produces a single, deterministic TEL text representation. Canonical serialization is defined only for documents that carry a schema — i.e. those whose semantic model exists. An untyped document (§8.2) has no semantic model and therefore no canonical form; its presentation model is its only stable representation.

Canonical serialization follows the same conventions as the `construct` operation (§22.2) for individual compounds, extended to the entire document:

- The document margin is zero.
- No interpreter directive is included.
- A pragma line is included, specifying the TEL version of the serializer and the schema identifier. The sigil is not specified in the pragma (the default # is used).
- When the schema is identified by both a URL and a signature (as a URL with a fragment, per §8.1), canonical serialization emits the **bare BASE-256-encoded signature** alone — the URL component is omitted. The signature is content-addressed and stable across resolver changes; a URL is a presentation-layer convenience that does not affect the semantic model. When the schema is identified only by URL (no signature was available at serialization time), the URL is emitted verbatim.
- No comments or remarks are included anywhere in the document.
- No tabulation lines are included; all compounds are serialized as ordinary (non-tabulated) lines.
- No blank lines appear between children at any level.
- The root node has no inline atoms (the document root is a virtual struct with no atom positions), so every root-level member is serialized as a compound child.
- At every non-root level, the **atom form escalation algorithm** below is applied to each Scalar value:
 1. **Inline atom** — used if **all** of the following hold:
 - the value contains no LF character;

- the value contains no run of two or more consecutive U+0020 SPACE characters (no hard space embedded in the value);
 - the value does not begin or end with U+0020 SPACE;
 - the value contains no occurrence of the document's sigil character preceded by U+0020 SPACE (which would be parsed as the start of a remark).
2. **Source atom** — used if the inline predicate fails **and** all of the following hold:
- no line of the value ends with U+0020 SPACE (no trailing spaces on any line);
 - the value contains no blank line (no run of two or more consecutive U+000A LF characters), which would terminate the source atom prematurely;
 - the value does not require byte-exact preservation of any character that source atoms cannot losslessly carry (notably leading SPACE on a line beyond the indentation).
3. **Literal atom** — used in every other case.

The **first** predicate the value satisfies determines the form; predicates further down the list are not consulted. This makes the choice deterministic across implementations even when a value would technically be representable in more than one form.

- Each inline atom uses a single preceding space (`precedingSpaces = 1`).
- Each compound child is indented by one level (two spaces) relative to its parent.
- Literal atoms use the delimiter `---` unless the payload contains that string as a line. In that case, the delimiter is lengthened by one – at a time (`----`, `-----`, ...) until the delimiter no longer appears as a line in the payload. This dash-extension algorithm is normative for canonical serialization; it is what makes property P3 (canonical determinism) hold for payloads that contain `---`.
- Line endings use LF mode.

Two documents with identical semantic models, serialized canonically by the same version of the specification, **MUST** produce identical text output.

22.4 Round-Trip Properties

The canonical text serialization (§22.3) and the BinTEL encoding (BinTEL §7) together with parsing and BinTEL decoding satisfy the following invariants. Conforming implementations **MUST** preserve these invariants; they are the contract between the spec and any tool that round-trips TEL data.

P1. Semantic round-trip via canonical text. For every well-typed semantic model *M* produced by parsing a TEL document under a schema *S*,

$$\text{type-assign}(\text{parse}(\text{canonical-serialize}(M, S)), S) \equiv M$$

That is: serialising *M* canonically and re-parsing under the same schema reproduces *M* exactly. Presentation-layer details that are not part of the semantic model — comments, remarks, tabulation, atom form, blank lines, sigil — are not required to round-trip; the semantic content is. In particular, a Scalar value carried as a source

atom round-trips through canonical serialisation as the same text string (per the LF-join rule of §14), so a multi-line scalar value is preserved byte-for-byte across the cycle.

P2. BinTEL round-trip. For every well-typed semantic model M and the same schema S ,

$$\text{bintel-decode}(\text{bintel-encode}(M, S), S) \equiv M$$

The BinTEL encoder (§7 of the BinTEL Specification) and decoder (§7.8) are mutual inverses on well-typed semantic models. As with P1, only the semantic content is preserved.

P3. Canonical-text determinism. For every well-typed semantic model M and schema S ,

$$\text{canonical-serialize}(M, S) = \text{canonical-serialize}(M, S)$$

(byte-equal). This follows from §22.3 and the fact that the schema fixes both member order and the atom-form-escalation rules. Together with P1, this means two distinct semantic models that round-trip-equal through canonical text **MUST** in fact be equal.

P4. BinTEL determinism. For every well-typed semantic model M and schema S ,

$$\text{bintel-encode}(M, S) = \text{bintel-encode}(M, S)$$

(byte-equal), which combined with the canonical child order of §7.2 (BinTEL spec) makes the **value hash** (§3 of the BinTEL Specification) a function of the semantic model alone — two implementations that agree on M and S **MUST** produce identical value hashes.

A conforming implementation that fails any of P1–P4 is non-conforming. A test suite for a TEL implementation **SHOULD** include cases that exercise each property over a representative set of documents (including pathological cases: multi-line scalar values requiring source/literal form, repeatable Fields with multiple atoms, layered schemas, exclude operations, and Reference cycles).

22.5 Concurrent Edit Composition

When two agents independently apply sequences of machine operations (§22.2) to the same baseline document, the resulting documents diverge. This subsection specifies a merge procedure that always produces a schema-valid result and exhibits strong eventual consistency for the semantic model. The procedure exploits TEL’s presentation-only constructs (remarks, attached comments — §11.1, §11.2) to record conflicts without distorting the semantic content.

Operation ordering. Every operation **MUST** carry a **Lamport timestamp** (a monotonically increasing integer per agent) and an **agent identifier** (a stable kebab-case string unique to the originating agent). The total order over operations is $(\text{timestamp}, \text{agent_id})$ — lexicographic on the pair, with `timestamp` compared as integers and `agent_id` as strings. This ordering is deterministic and depends only on the operation set, not on the order of arrival.

Merge function. Given a baseline document B_0 and two operation sequences O_A , O_B each derived from B_0 , the merge produces (D, R) :

1. Form the combined ordered sequence $S = \text{sort}(O_A \cup O_B)$ by the order above.
2. Starting from B_0 , process each operation op in S :
 1. Determine whether op can be applied to the current document: its target (path or keyword) must still resolve, and applying it must leave the document schema-valid (no E2xx or E3xx errors against the schema in effect for the document).
 2. If yes, apply op normally.
 3. If no, **demote** op : do not apply it; instead, attach a **conflict remark** to the affected compound (or, if the target was removed, a **conflict comment** to the nearest surviving ancestor block) describing what was attempted. The remark or comment records the operation's agent, timestamp, and a short description.
3. Return D (the resulting document) and R (the list of demoted operations, for downstream audit).

Conflict remark format. When a demoted operation's target still exists, a remark is attached to the target compound:

```
<sigil> Merge conflict (agent <agent_id>, ts=<timestamp>):  
<operation summary>
```

When the target has been removed by a prior operation in the merge, a comment is attached to the nearest surviving block:

```
<sigil> Merge conflict (agent <agent_id>, ts=<timestamp>):  
<operation summary> – target was removed
```

In both formats, `<sigil>` is the document's effective sigil as determined by §8.3 — not a hard-coded `#`. Merge engines **MUST** use the document's sigil when writing conflict markers; using `#` against a document whose sigil is (say) `;` would emit a compound keyword `#` and trigger **E306** at parse time rather than producing a valid remark. For example, if a document's sigil is `;`, a conflict remark attached to a compound at indent 1 is written as:

```
keyword atoms ; Merge conflict (agent alice, ts=42): update-  
value attempted
```

and a free-standing conflict comment whose target was removed appears as:

```
; Merge conflict (agent alice, ts=42): update-value – target was  
removed
```

Properties.

- **Total.** The merge function is total: it always returns a (D, R) . There is no “merge failed” outcome at the document level. Pathological inputs are recorded as conflict remarks; the document remains schema-valid.
- **Convergent.** Two agents that have observed the same set of operations produce identical canonical text (§22.3), because §22.3 strips remarks and comments. The presentation layer may differ in the number and placement of remarks, but the semantic content is identical.

- **Forensic.** Every operation that could not be applied is recorded as a remark or comment carrying the agent, timestamp, and a description. An auditor or a later editor can review the conflicts and apply them manually.
- **Schema-driven.** The “would this leave the document schema-valid?” check uses the existing §20 / §21 error machinery; no new validation surface is needed.

Operation subset compatibility. The merge procedure is well-defined for all §22.2 operations. For some operations, conflicts are particularly common:

- Concurrent update–value on the same compound: the lower-ordered operation applies; the other is demoted.
- Concurrent delete and update–value on overlapping subtrees: the delete wins; the update–value is demoted.
- Concurrent insert–before / insert–after at the same position: both succeed (each inserts a new element); their relative order in the resulting document is determined by the Lamport order of the two operations.
- Concurrent reorder–* and replace: typically demoted to a remark on the parent block because the target may be in flux.

A schema-aware agent MAY pre-emptively coordinate to avoid conflicts (locking, lease-based ownership, etc.), but this specification does not mandate any coordination protocol. Coordination is the application’s concern; the merge procedure above is the contract for the case where coordination fails or is absent.

23. Invalidity Conditions

A TEL document is invalid if any condition identified by a **E1xx** or **E3xx** error code in this specification is triggered. A schema is invalid if any **E2xx** condition is triggered. The complete taxonomy of all error conditions, their trigger sections, and their recovery strategies are given in §19.3 and §19.5 respectively.

24. Formal Type System and Subtyping (Informative)

This section is **informative**, not normative. It gives a formal account of TEL’s type system, defines a subtype relation $<:$ over the types of §20, states the inference rules for that relation, sketches that the layer composition rules of §20.3 produce subtypes of the base schema, and shows that the **Liskov Substitution Principle** holds: a document valid under a subtype is, after projection, a valid document under the supertype.

Conformance to this specification does not require an implementation to compute $<:$ explicitly: §20.2, §20.3, and §21 are stated as concrete algorithms and discharge every constraint a conforming parser must check. §24 exists to give schema authors and tool builders a precise shared vocabulary for what “compatible schemas” means.

24.1 Type Grammar

The recursive type structure of §20, written compactly:

```

T ::= Struct(M*, V*)           – record / product
    | Scalar(V*, d?)          – leaf value
    | Flag                     – presence-only
    | Reference(N)             – named recursive type

M ::= Field(K, r, p, T)
    | Select(r, p, X+)

X ::= Variant(K, T)

K, N, V ::= identifier         (per §20.7, with optional leading
primes)
d        ::= text              (default value, optional)
r, p     ::= true | false      (required, repeatable)

```

A **schema context** Δ is a finite map from Definition names to Definition bodies:

$$\Delta : N \rightarrow (M^*, V^*)$$

So $\Delta(N) = (\text{members}, \text{validators})$ when the schema has record $N \backslash n \dots \backslash n$ validate ... (or analogously the validators list alone for a scalar N Definition). The composed Δ is the merge of the base schema's `Schema.types` $\cup$ `Schema.scalars` with each layer's `Layer.types` $\cup$ `Layer.scalars`, per §20.3.

The `Exclude(K)` member of §20.3 is a layer-only construct that operates on Δ during composition; it does not appear in a composed type and so is not part of T .

24.2 Membership

The membership judgment $\Delta \vdash d : T$ is read “under context Δ , semantic-model element d is of type T ”. It is defined by induction on T :

[Mem-Flag] "present")	(no premise; Flag has only $\Delta \vdash \text{flag-node} : \text{Flag}$
[Mem-Scalar] text returns Valid.	For every v in V^*, validator v applied to text returns Valid. $\Delta \vdash \text{scalar-node}(\text{text}) : \text{Scalar}(V^*, d?)$
[Mem-Struct] §20.2 atom + compound contiguity). returns Valid.	d has children $c_1, \dots, c_n$ matching M^* per phases (member-fill, required, repeatable, contiguity). For each c_i, $\Delta \vdash c_i : \text{type-of-}c_i$. For every v in V^*, validator v applied to d $\Delta \vdash \text{struct-node}(c_1, \dots, c_n) : \text{Struct}(M^*,$ $V^*)$
[Mem-Reference]	$\Delta \vdash d : \text{Struct}(\text{members}, \text{validators})$

$\Delta(N) = (\text{members}, \text{validators})$

$\Delta \vdash d : \text{Reference}(N)$

These rules correspond exactly to the type-assignment algorithm of §20.2 and the validation rules of §21. A document is valid under a schema iff $\Delta \vdash d : \text{Schema.document}$ (treating the schema-document Struct as the root type).

24.3 Subtype Relation

The subtype relation $T_1 <: T_2$ (under Δ , when needed) is defined by the following inference rules. The intuition is: $T_1 <: T_2$ means any element of type T_1 contains enough information to satisfy any consumer that expects type T_2 .

[Sub-RefI]	$T <: T$	
[Sub-Trans]	$T_1 <: T_2 \quad T_2 <: T_3$	
	$T_1 <: T_3$	
[Sub-Flag]	$\text{Flag} <: \text{Flag}$	
[Sub-Scalar]	$V_2 \subseteq V_1$	(sub has at least super's validators)
	$\text{Scalar}(V_1, d_1) <: \text{Scalar}(V_2, d_2)$	
[Sub-Struct]	For every member $m_2 \in M_2$, there exists $m_1 \in M_1$ such that $m_1 <:_M m_2$.	
	$V_2 \subseteq V_1$	(sub has at least super's validators)
	$\text{Struct}(M_1, V_1) <: \text{Struct}(M_2, V_2)$	
[Sub-Ref-L]	$\Delta \vdash \text{Struct}(\Delta(N).\text{members}, \Delta(N).\text{validators}) <: T$	
	$\Delta \vdash \text{Reference}(N) <: T$	
[Sub-Ref-R]	$\Delta \vdash T <: \text{Struct}(\Delta(N).\text{members}, \Delta(N).\text{validators})$	
	$\Delta \vdash T <: \text{Reference}(N)$	

Member subtyping ($<:_M$) has two cases, one per member kind:

[Sub-Field]	$K_1 = K_2$	(same keyword)
	$T_1 <: T_2$	(covariant in type)
	$r_2 \Rightarrow r_1$	(sub at least as required)
	$p_1 \Rightarrow p_2$	(sub at least as required)

most as repeatable)

T_2)

[Sub-Select]

$\text{Variant}(K, T_2) \in X_2$

variants $\subseteq$ super's variants)

least as required)

most as repeatable)

$\text{Field}(K_1, r_1, p_1, T_1) <:_M \text{Field}(K_2, r_2, p_2,$

For every $\text{Variant}(K, T_1) \in X_1$, there exists

such that $T_1 <:_M T_2$.

$r_2 \Rightarrow r_1$

$p_1 \Rightarrow p_2$

$\text{Select}(r_1, p_1, X_1) <:_M \text{Select}(r_2, p_2, X_2)$

(sub's

(sub at

(sub at

Reading the rules.

- **Records are subtyped by extension.** A Struct with more members is a subtype: every member required by the supertype must be present (and subtype-compatible) in the subtype; the subtype may have additional members the supertype knows nothing about.
- **Sums are subtyped by narrowing.** A Select with fewer variants is a subtype: every variant offered by the subtype must be offered by the supertype, but the supertype may accept variants the subtype never produces.
- **Scalars are subtyped by tightening.** A Scalar with more validators is a subtype: values that satisfy the subtype's validators automatically satisfy the supertype's.
- **required is tightenable.** Going from `required: false` to `required: true` is the subtype direction: any value satisfying the stricter rule also satisfies the lax one.
- **repeatable is tightenable in the opposite direction.** Going from `repeatable: true` to `repeatable: false` is the subtype direction: a non-repeatable cardinality (0 or 1) is a special case of a repeatable cardinality (0 or more).

The rules are sound: each premise mirrors a constraint that would distinguish valid elements of one type from valid elements of the other. Reflexivity and transitivity are immediate from the structure.

24.4 Layer Composition Produces Subtypes

Theorem (Layer Subtyping). Let S_{base} be a schema with document Struct D_0 and Definition context Δ_0 . Let L be a layer satisfying the validity constraints of §20.3. Let (D_1, Δ_1) be the result of applying L to (D_0, Δ_0) per §20.3. Then:

$\Delta_1 \vdash D_1 <:_M D_0$ (with Δ_0 viewed as a subset of Δ_1)

That is, **applying a layer always produces a subtype of the base.**

Proof sketch. §20.3 permits seven operations:

1. **Add Field** — D_1 has all of D_0 's members plus a new Field. By [Sub-Struct], $D_1 <: D_0$ because every member of D_0 still appears in D_1 (with identical $<:_M$ -self).
2. **Add Select** — As above, additional Select member. $D_1 <: D_0$.
3. **Definition merge (recursive Struct extension)** — when a layer adds a Field whose keyword matches an existing Field of Struct type, the resulting Struct's members are the union of base and layer. By [Sub-Struct] applied recursively, the new Struct is a subtype of the old, so the Field is subtype-compatible by [Sub-Field].
4. **Exclude variant** — D_1 's Select has strictly fewer variants than D_0 's. By [Sub-Select], $D_1 <: D_0$.
5. **Tighten required** — D_1 has the same Field/Select but with `required: false` $\rightarrow$ `true`. By [Sub-Field] / [Sub-Select] (premise $r_2 \Rightarrow r_1$), the tightening is subtype-producing.
6. **Tighten repeatable** — D_1 has the same Field/Select but with `repeatable: true` $\rightarrow$ `false`. By [Sub-Field] / [Sub-Select] (premise $p_1 \Rightarrow p_2$ where the subtype is the one with `p1 = false`), the tightening is subtype-producing.
7. **Add struct or scalar validator** — $V_1 \supseteq V_0$. By [Sub-Struct] or [Sub-Scalar], the type is a subtype.

§20.3 forbids the supertype-producing operations: removing a Field, adding a variant to an existing Select, loosening required from true to false (E215), loosening repeatable from false to true (E216), dropping a validator. By construction, no permitted layer operation moves in the supertype direction.

By transitivity ([Sub-Trans]), the iterative application of layers yields a chain of subtypes:

$$D_n <: D_{\{n-1\}} <: \dots <: D_1 <: D_0$$

The composed schema is a subtype of the base. ■

24.5 The Liskov Substitution Theorem

Theorem (LSP). If $T_1 <: T_2$ and $\Delta \vdash d : T_1$, then there exists a **projection** $\pi_{\{T_2\}}(d)$ such that $\Delta \vdash \pi_{\{T_2\}}(d) : T_2$.

Projection discards the parts of d that aren't addressable from T_2 :

$\pi_{\{T_2\}}(d) = \text{case } (T_2, d) \text{ of:}$

Flag, flag-node	$\rightarrow$ flag-node
Scalar($V_2, _$), scalar-node(text)	$\rightarrow$ scalar-node(text)
	(validators in T_2 are checked separately)
Struct(M_2, V_2), struct-node(c^*)	$\rightarrow$ struct-node(c'^*) where c'^* is
	{ $\pi_{\{\text{type-of-}m_i\text{-in-}T_2\}}(c_i)$
	c_i is a child whose
keyword matches some $m_2 \in M_2$ }	
Reference(N), d	$\rightarrow \pi_{\{\text{Struct}(\Delta(N).members,$
$\Delta(N).validators\}}(d)$	

In words: at every Struct, drop children whose keywords don't appear in T_2 's members; recurse into the surviving children at the type T_2 ascribes to them. Scalars and Flags pass through unchanged.

Proof sketch. By induction on T_2 :

- **Flag.** π is the identity. The result is the same flag-node, of type Flag. ✓
- ****Scalar(V_2 , $_$).** π is the identity on the text. By [Sub-Scalar], $V_2 \subseteq V_1$; since d satisfied every validator in V_1 , it satisfies every validator in V_2 (subset). So $\pi_{\{\text{Scalar}(V_2)\}}(d) : \text{Scalar}(V_2)$. ✓
- **Struct(M_2 , V_2).** For each $m_2 \in M_2$, [Sub-Struct] gives a matching $m_1 \in M_1$ with $m_1 <:_M m_2$. The corresponding child in d has a type that's a subtype of $\text{type-of-}m_2$ by [Sub-Field] or [Sub-Select]. By IH on the child, $\pi_{\{\text{type-of-}m_2\}}(\text{child}) : \text{type-of-}m_2$. Validators in $V_2 \subseteq V_1$ were satisfied by d . Required/repeatable constraints carry: if m_2 required the member, m_1 did too, so the member is present; if m_2 allowed non-repeated, m_1 was non-repeatable, so the constraint holds. ✓
- **Reference(N).** Inductive: substitute the resolved Struct. ✓

In every case the projection yields a valid T_2 element. ■

24.6 Consequences for Tooling

LSP gives the schema ecosystem a useful guarantee:

- A **document written against a subtype schema** can be consumed by any tool that understands the supertype schema, as long as the tool reads through the supertype's schema (so it implicitly performs the projection by ignoring unknown fields).
- The **schema-signature compatibility rule** in §8.2 of the TEL Specification (a signature A is compatible with signature B when A 's hash sequence is a subsequence of B 's) is now grounded: §24.4 establishes that the composed-with-fewer-layers schema is a supertype of the composed-with-more-layers schema, so a document written against the longer composition can be consumed (after projection) by a tool expecting the shorter composition.
- **construct operations** (§22.2) can target the supertype's schema: a freshly constructed compound that satisfies the supertype's required-set is automatically a valid subtype value at any position where the subtype permits the same members. (The reverse — using a supertype-validated compound at a subtype position — is NOT generally safe; the subtype may demand additional fields the supertype didn't supply.)
- **The implementation never needs to compute $<:$ explicitly.** The type-assignment algorithm (§20.2) directly checks element membership; the layer composition algorithm (§20.3) directly produces the subtype. Subtyping is a property of how these algorithms fit together, not a separate runtime check.

24.7 What This Type System Does NOT Cover

- **Variance of validators.** The subtype relation requires $V_2 \subseteq V_1$ — the subtype has at least the supertype's validators. It does not require that any validator *strictly* tightens the value space (a validator that always returns Valid satisfies the subset relation trivially). A schema author who wants a meaningful subtype

refinement is expected to add validators that genuinely restrict; the type system does not check this.

- **Cross-document subtyping.** Two distinct schemas (different base names) are not subtypes of each other under this relation, even if their structural shapes happen to match. Subtyping is meaningful only within a single base-schema family (the same `Schema.name`, plus an ordered subset of layers).
- **Behavioural / semantic subtyping.** The relation here is purely structural. Two Scalars with the same validators are subtype-equivalent even if their *names* suggest different semantics (postal-code vs phone-number). Behavioural distinctions are the application's concern.

25. Specification Status

This v1.0 specification is complete for single-document and single-agent use. The error taxonomy (**E101–E123** parsing, **E201–E217** schema, **E301–E311** validation) is contiguous; every code is referenced at the point in the body where its trigger condition is defined and appears exactly once in the diagnostic tables of §19.3, §20.1, and §21.6. Worked examples — including TEL documents shown with their presentation model, semantic model, and BinTEL byte sequence — are recorded in [demo/](#). Round-trip properties (P1–P4) are stated in §22.4. Concurrent-edit composition is stated in §22.5. Schema compatibility is defined by the subtype relation of §24 and decided on signatures per §8.2.